

基于粒子群算法的烟幕干扰弹投放策略研究

摘 要

烟幕干扰弹在防御领域凭借其高精准度的特点而被应用在对特定目标进行遮蔽的任务中。本文针对无人机投放烟幕干扰弹遮蔽导弹视野的问题，主要建立了针对性优化的粒子群算法，求解得到了无人机飞行速度、烟幕干扰弹投放及引爆位置等关键数据，达到了使综合遮蔽真目标时间最长的效果。

针对问题一，为求解有效遮蔽时长问题，本部分首先以严谨的数学证明论证了在导弹视角下，连续的真目标表面可以被简化为有限个特征点。而判断其遮蔽效果的问题，可进一步简化为计算二维向量的问题。再通过物理计算实时运动状态，方便将连续的时间微元化进行积分逼近，大大简化了模型的计算。得到了有效遮蔽时长1.391642 s。

针对问题二，为使得有效遮蔽时长最大化，本部分以关键求解数据为决策变量建立目标函数。结合数学验证与实际测试，得到了目标函数并非凸函数、极容易陷入局部最优解、边界值可能为最优解的结论，因而采用了粒子群算法（PSO）并对其进行了针对性优化。在传统粒子群算法上，本文创新性地进行了针对性优化：停滞时择机重启、动态调整惯性权重、对边界特殊处理。最终模型运行得到了最大有效遮蔽总时长4.588844 s。详细结果参见公式(17)。

针对问题三，在问题二的基础上，先将额外的决策变量及约束条件加入至模型中，再通过将所有决策变量统一为一个优化向量的方法简化计算难度，并采用针对性优化的粒子群算法（APSO）进行一体化求解。本部分中算法将直接以去重并集的总时长为优化目标，以快速找到最优的协同策略组合。最终模型运行得到了最大有效遮蔽总时长7.645346 s。详细结果参见公式(29)及 **result1.xlsx**。

针对问题四，在问题二的基础上，增加的决策变量数量较多，这使得模型优化维度极高且存在策略耦合难以求解。本部分以“贪心+降维”的思路分阶段执行，依次选出对当前状态改善效果最佳的解并固定保留。而对于较低维度子问题的求解，则采用性能更强的多岛屿异质粒子群算法（Multi-Island PSO）。最终模型运行得到了最大有效遮蔽总时长11.694278 s。详细结果参见公式(44)及 **result2.xlsx**。

针对问题五，在第四问“贪心+降维”模型的基础上，建立了以最坏可见性为判据的遮蔽模型，通过引入投弹时序权重因子，增大后期投掷的烟幕干扰弹对于整体有效遮蔽时间的效用增益，避免贪心算法因过度优化前期投弹而忽视后期遮蔽补位价值，强制算法在迭代过程中对其计算与优化。最终模型运行得到了最大有效遮蔽总时长22.195517 s，详细结果参见 **result3.xlsx**。

本文研究了无人机投放烟幕干扰弹实现导弹遮蔽的优化问题。通过数学建模与证明，将复杂遮蔽判定简化为二维向量计算；以有效遮蔽时长为目标函数，针对非凸优化难题引入并改进粒子群算法，结合动态权重、边界处理及多岛屿策略，逐步解决多变量耦合问题，在不同阶段模型中，还引入贪心+降维及时序权重因子，提升整体遮蔽效果。最终得到最大化遮蔽时长的协同优化方案。

关键词：烟幕干扰弹，粒子群算法，二分修正，贪心算法，评估函数

一、问题重述

烟幕干扰弹具有低成本、高效能、可形成烟幕或气溶胶云团的特点，使其在安全领域起到干扰导弹的作用。通过无人机精准投放、精确控制起爆时间的方式，烟幕干扰弹可以在指定位置与指定时间形成一定半径的球状烟幕云团，在一定时间内可以对目标形成有效遮蔽。

为了保护一真目标免受导弹攻击，我们需要建立数学模型解决以下问题：

问题一：为对一枚导弹进行干扰，现利用初始位置已知、速度为 120 m/s 、方向指向假目标的一架无人机投放一枚烟幕干扰弹。烟幕干扰弹投放于受领任务的 1.5 s 后，并起爆于投放后的 3.6 s 。计算烟幕干扰弹有效遮蔽导弹的时长。

问题二：为对一枚导弹进行干扰，现利用一架无人机投放一枚烟幕干扰弹。为使烟幕干扰弹有效遮蔽导弹的时长最大化，确定最佳的无人机速度大小及方向、烟幕干扰弹的投放及起爆点。

问题三：在问题二的背景下，唯一一架无人机可以有间隔地投放三枚烟幕干扰弹，仍然要使有效遮蔽导弹的时长最大化。需要确定最佳的无人机速度大小及方向、三枚烟幕干扰弹分别的投放及起爆点，并分别求出有效遮时长。

问题四：在问题二的背景下，无人机的数量增长至三架，仍然要使有效遮蔽导弹的时长最大化。分别确定无人机最佳的速度大小及方向、烟幕干扰弹最佳的投放及起爆点，并分别求出有效遮蔽时长。

问题五：在问题二的背景下，无人机、每架无人机的烟幕干扰弹存量、导弹数目分别提升至五架、至多三枚、三枚，仍然要使有效遮蔽导弹的时长最大化。分别确定所有无人机最佳的速度大小及方向、所有烟幕干扰弹最佳的投放及起爆点，并分别求出所有烟幕干扰弹有效遮蔽时长。

二、问题分析

2.1 重要支撑性论证

● 命题一：

从一个点光源对球体作投影，投到平面上的阴影（轮廓）是椭圆。

● 命题二：

从点光源把一个平面内的圆投影到与该圆平行的另一个平面上，投影轮廓仍然是圆。

● 命题三：

如果一个两个相离的圆均在同一椭圆内，则这由两个圆以及二者的两条外公切线包络形成的形状也完全在该椭圆内。

可以得到结论：只要圆柱的上下表面圆周均未出现在导弹视野中，即可保证整个圆柱都无法观察到，此时则可以认为真目标被有效遮蔽。

上述命题详细证明过程参见 附录 2。

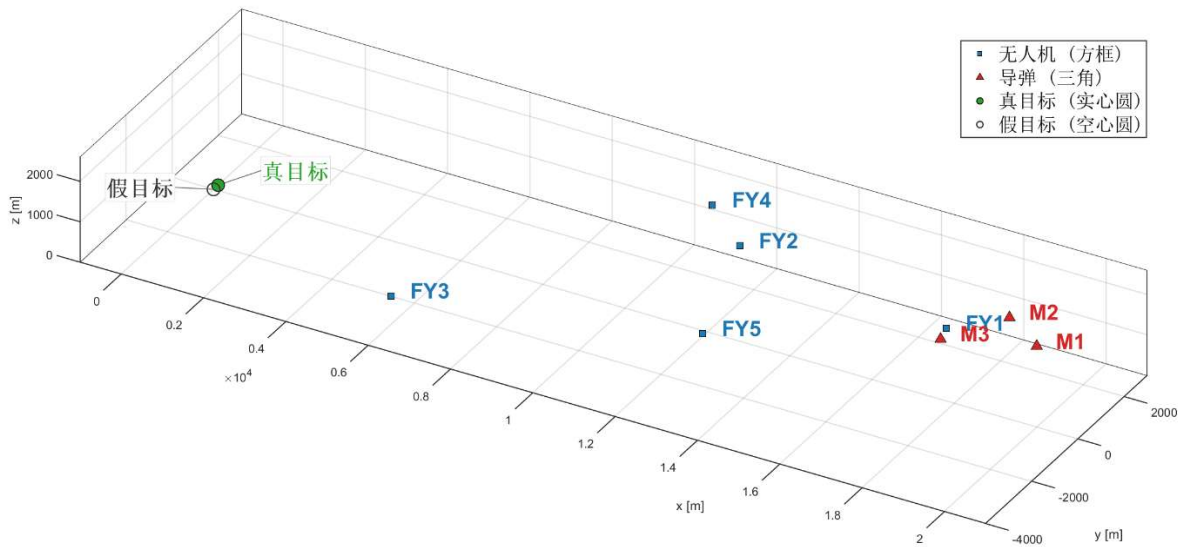


图 1 初始位置示意图

2.2 问题一的分析

问题一要求在给定条件下，计算烟幕干扰弹对导弹的有效遮蔽时长。本题的难点在于建立有效遮蔽的判定条件。由于真目标是圆柱体，表面的点无限多且难以表示，可以考虑通过 2.1 的几何关系转化为对上下表面外轮廓圆的判断，采用离散化和几何逼近的思想^[2]来解决。最终通过扫描整个时间域，累加所有满足条件的时间微元，即可求得总时长。

2.3 问题二的分析

问题二是一个寻找最优解的问题，要求确定无人机 FY1 的最优飞行、投弹及引爆策略，使遮蔽导弹 M1 的时间最大化。本题是一个决策变量不唯一且约束条件复杂的非线性优化问题。

考虑采用 APSO 进行求解。可以在算法迭代过程中使用一个快速的评估函数来指导搜索方向，以提高计算效率。搜索时可以采用停滞重启、动态调整惯性权重、对边界进行特殊处理的方式来防止算法陷入局部最优解。得到结果后，再代入问题一中的模型进行复核，以确保结果的合理性、准确性。

2.4 问题三的分析

问题三在问题二的基础上，要求为单架无人机规划三枚烟幕干扰弹的协同投放策略，目标是使对导弹 M1 的总有效遮蔽时长最大化，即所有独立遮蔽时间与多枚烟幕干扰弹共同遮蔽的并集。本题的难点在于决策变量的增多、约束条件的增加会导致搜索空间呈指数级增长。

计划将所有决策变量统一为一个优化向量，继续采用 APSO 进行一体化求解。在适应度函数的设计中，会将投放间隔约束作为关键的可行性判断，对违反该约束的解施加惩罚。算法将直接以去重并集的总时长为优化目标，以找到最优的协同策略组合。

2.5 问题四的分析

问题四在问题二的基础上，要求为三架不同初始位置的无人机各规划一枚烟幕弹的投放策略，目标是最大化三枚弹协同作用下的总有效遮蔽时长（并集）。本题的难点在于优化维度极高且存在策略耦合。由于三架无人机的飞行和投弹参数相互独立，总决策变量高达 12 维而难以计算。同时，各弹的遮蔽效益并非简单叠加，其边际贡献取决于另外两枚弹的投放时空位置，需要高效的协同策略。

计划采用贪心算法结合智能优化的降维思想。我们将这个 12 维的复杂问题分解为三个串行的 4 维优化子问题。算法分阶段执行：首先选出一架无人机并固定其策略；之后依次在剩余无人机中，选择能使当前边际遮蔽增益最大的一个并固定其策略。对于每个 4 维子问题的求解，则采用性能更强的多岛屿异质粒子群算法（Multi-Island PSO），以保证各阶段决策的质量。

2.6 问题五的分析

问题五是一个建立在问题二的基础上的大规模、多目标的资源分配问题。要求为 5 架无人机规划总计最多 15 枚干扰弹的投放策略，以同时应对 3 枚来袭导弹，实现整体拦截效果最优。本题的难点在于决策空间巨大，且目标不再是单一的总时长最大化，而是需要在三枚导弹的防护效果之间取得平衡，避免因单一导弹防护过弱导致整体任务失败，这是一个典型的多目标优化难题。

计划采用串行贪心决策的框架，将 15 枚弹的复杂部署分解为 15 个独立的单弹最优决策子问题。其核心创新在于引入“公平性”适应度函数，该函数通过对数效用函数，加权了为防护薄弱导弹提供掩护所带来的收益。在每个决策步骤，算法将并行评估所有无人机可执行的“下一步动作”，并选择“公平性”评分最高的方案予以执行，以此在“总时长”和“均衡防护”两个目标间取得平衡。

三、模型假设

- 假设一：烟幕干扰弹的投掷过程、爆开过程不考虑空气阻力。
- 假设二：导弹可视为质点，具有以自身为中心的球形视野，真目标的任意部分出现在导弹的视野中即认为被导弹发现，为无效遮挡。
- 假设三：烟幕干扰弹爆开后形成一个以3m/s的速度下沉且相对位置不变。
- 假设四：任务下发至无人机以及无人机开始执行受领任务均在瞬时完成。

四、符号说明

表 1 本文的符号说明

符号	说明	单位
$i \in \{1, 2, 3, 4, 5\}$	无人机编号	-
$j \in \{1, 2, 3\}$	烟幕干扰弹编号	-
$k \in \{1, 2, 3\}$	导弹编号	-
$n \in \{1, 2, ..., 720\}$	取样点编号	-

(x_{Mi}, y_{Mi}, z_{Mi})	导弹坐标	m
$(x_{FYi}, y_{FYi}, z_{FYi})$	无人机坐标	m
v_M	导弹速率	m/s
v_{FYi}	第 <i>i</i> 架无人机的速率	m/s
t_{FYij}	第(<i>i,j</i>)枚烟幕干扰弹被投放前的飞行时间	s
t_{Bij}	第(<i>i,j</i>)枚烟幕干扰弹的投放至引爆的时间间隔	s
$t_{ij} = t_{FYij} + t_{Bij}$	第(<i>i,j</i>)枚烟幕干扰弹的受领任务至引爆的时间	s
v_D	烟幕下沉速率	m/s
$L_k = \sqrt{x_{Mk}^2 + y_{Mk}^2 + z_{Mk}^2}$	第 <i>k</i> 枚导弹与假目标之间的距离	m
θ_i	第 <i>i</i> 架无人机运动的方位角	度
r	真目标圆柱半径	m
R	烟幕半径	m

五、模型建立与求解

5.1 问题一模型的建立与求解

5.1.1 问题一模型^[3]的建立

由于无人机在接收到任务后，瞬时调整运动策略，以方位角 θ_i ，运动 t_{FYi} 后投放烟幕干扰弹，该运动过程完成后，无人机的位置为

$$(x_{FYi} + v_{FYi}t_{FYi}\cos\theta, y_{FYi} + v_{FYi}t_{FYi}\sin\theta, z_{FYi}) \quad (1)$$

此时无人机投放烟幕干扰弹，在不考虑空气阻力影响的情况下，烟幕干扰弹将近似做平抛运动，在投放后至起爆前的时间间隔中，烟幕干扰弹下落高度为

$$\Delta h = \int_0^{t_B} gt \, dt = \frac{1}{2}gt_B^2 \quad (2)$$

起爆时，烟幕干扰弹的位置为

$$\left(x_{FYi} + v_{FYi}t_i\cos\theta, y_{FYi} + v_{FYi}t_i\sin\theta, z_{FYi} - \frac{1}{2}gt_B^2 \right) \quad (3)$$

导弹的位置为

$$\left(x_{Mi} \frac{L_i - v_M t_i}{L_i}, y_{Mi} \frac{L_i - v_M t_i}{L_i}, z_{Mi} \frac{L_i - v_M t_i}{L_i} \right) \quad (4)$$

由先前证明可知，讨论烟幕干扰弹是否对真目标进行了有效遮蔽等价于讨论在导弹视角下能否观察到真目标的上下表面的轮廓。因此，我们采用了分别在上下表面的外轮

廓进行**等差极角取点**来模拟整个上轮廓与下轮廓的形状。

即上轮廓有点集 $\left\{(r \cos \varphi_i, 200 + r \sin \varphi_i, 10) \mid \varphi_i = \frac{1}{180} \pi, \frac{2}{180} \pi, \dots, 2\pi\right\}$ ，下轮廓有点集 $\left\{(r \cos \varphi_i, r \sin \varphi_i, 10) \mid \varphi_i = \frac{1}{180} \pi, \frac{2}{180} \pi, \dots, 2\pi\right\}$ 。

在时间区间 $[t_i, t_i + 20]$ 内，烟幕干扰弹10 m 范围内均能够形成有效遮蔽，并始终以3 m/s 的速度向下移动，对于时间区间 $[t_i, t_i + 20]$ ，有导弹位置

$$P_A = \left(x_{M_i} \frac{L_i - v_M(t_i + t)}{L_i}, y_{M_i} \frac{L_i - v_M(t_i + t)}{L_i}, z_{M_i} \frac{L_i - v_M(t_i + t)}{L_i} \right) \quad (5)$$

烟幕中心点位置

$$P_C = \left(x_{FY_i} + v_{FY_i} t_i \cos \theta, y_{FY_i} + v_{FY_i} t_i \sin \theta, z_{FY_i} - \frac{1}{2} g t_i^2 - v_D t \right) \quad (6)$$

以及选定的采样点 Q_n 的位置为 P_B 。

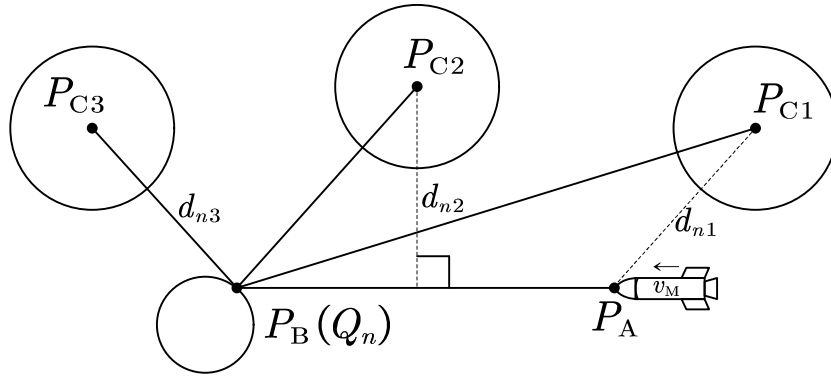


图 2 d 取值分类讨论示意图

此时可以认为 $\overrightarrow{P_A P_B}$ 即为导弹视角下的“视线”向量^[4]，如图所示，令 $u = \frac{\overrightarrow{AB} \cdot \overrightarrow{AC}}{\|\overrightarrow{AB}\|^2}$ ，

此时 $\overrightarrow{P_A P_B}$ 与 P_C 的距离 d_n 为

$$d_n = \begin{cases} |\overrightarrow{P_A P_C}|, & u \in (-\infty, 0) \\ |\overrightarrow{P_C P_D}| = \left| \overrightarrow{P_A P_C} - \frac{\overrightarrow{P_A P_B} \cdot \overrightarrow{P_A C}}{|\overrightarrow{P_A P_B}|^2} \times \overrightarrow{P_A P_B} \right|, & u \in [0, 1] \\ |\overrightarrow{P_A P_B}|, & u \in (1, \infty) \end{cases} \quad (7)$$

考虑寻找离烟幕中心最远的视线，即寻找 $\max(d_n)$ ，此时若 $R \geq \max(d_n) \geq d_n$ ，即可认为导弹视线被有效遮蔽。

5.1.2 问题一模型的求解

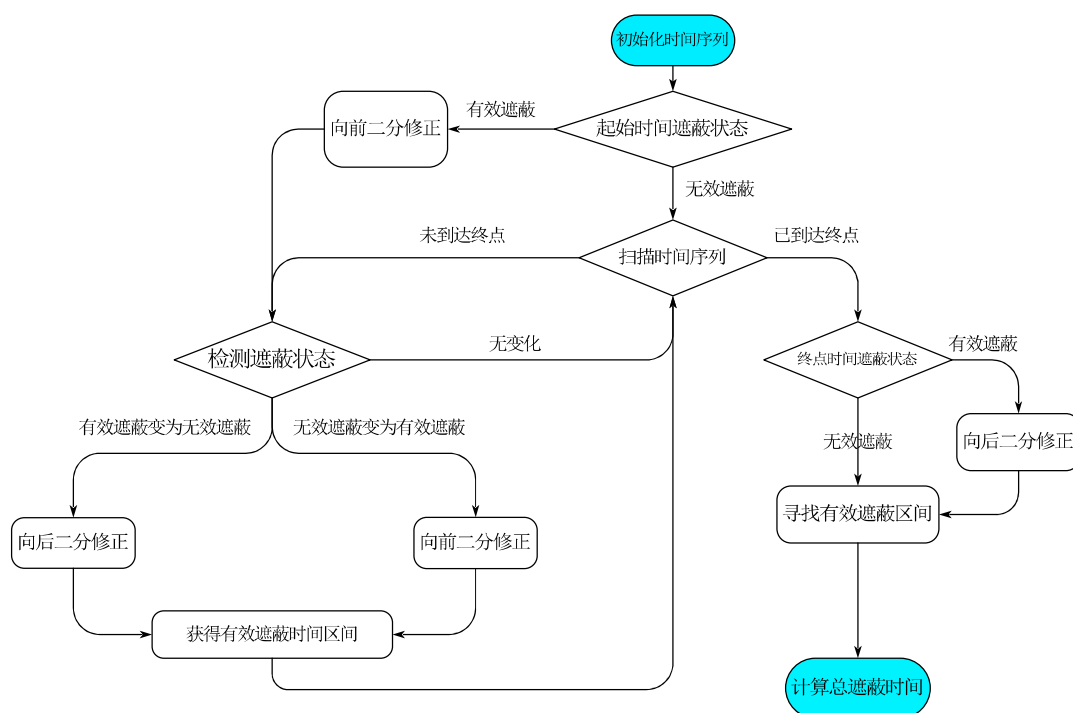


图3 有效遮蔽状态扫描与二分法精修遮蔽空间示意图

如 图 3 所示，在问题一中，只需要对时间序列进行扫描，把所有有效遮蔽的时间微元做积分运算，即可获得总有效遮蔽时长。在算法优化层面，考虑到有效遮蔽时间大多为连续的，可以采取二分修正的方式，快速锁定时间区间的边界，获得有效遮蔽的时间，有利于提高多次迭代更新的速度。在达到时间序列终点时停止扫描，此时已获得的有效遮蔽区间即为题目所需。

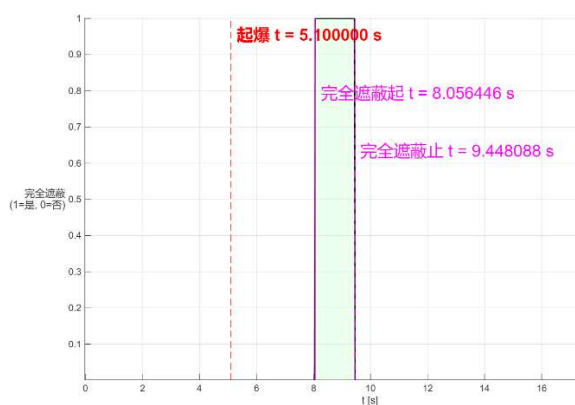


图4 有效遮蔽时序图

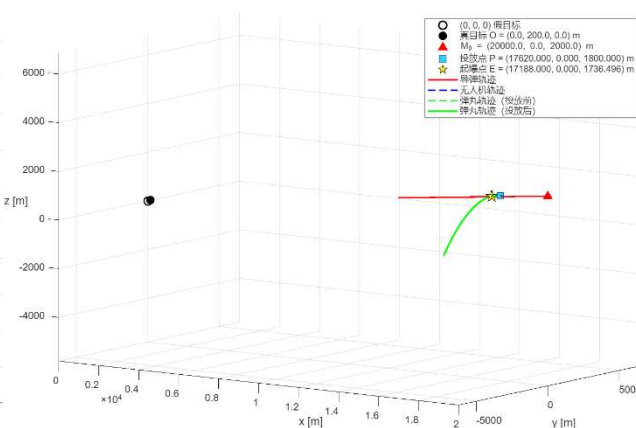


图5 运动轨迹示意图

如 图 4 、 图 5 所示，模型最终计算结果为

$$\text{总有效遮蔽时长 } T_{\text{cov}} = 1.391642 \text{ s} \quad (8)$$

5.2 问题二模型的建立与求解

5.2.1 问题二模型的建立

本部分建立了使烟幕干扰弹有效遮蔽时长最大的优化模型。在在遮蔽有效性等约束条件下，确定最佳的无人机飞行时长、速率、方向以及烟幕干扰弹引爆时间，实现使烟幕干扰弹有效遮蔽时长最大的目标。

● 决策变量

欲使最终有效遮蔽时间最长，可以采用粒子群算法针对无人机运动方位角 θ 、定时引爆时间 t_B 、无人机的速度 v_{FY} 与起爆时间 t 进行非线性优化，寻找使有效遮蔽时间最长的 θ 、 t_B 、 v_{FY} 、 t 的具体数值解。因此，**决策变量为 θ 、 t_B 、 v_{FY} 、 t 。**

● 目标函数

$$\max T(\theta, t_B, v_{FY}, t) = \int_0^{t_{\text{stop}}} f(t) dt \quad (9)$$

● 约束条件

1. **方位角约束：**方位角取值范围为 $[0^\circ, 360^\circ)$ ，即

$$\theta \in [0^\circ, 360^\circ) \quad (10)$$

2. **引爆时间约束：**烟幕干扰弹引爆时间不能早于投放，不能晚于坠落，即

$$0 \leq t_B \leq \sqrt{\frac{2z_{FY}}{g}} \quad (11)$$

3. **飞行时间约束：**无人机飞行时长非负，即

$$t_{FY} \geq 0 \quad (12)$$

4. **飞行速度约束：**根据题目要求，无人机的飞行速度有限制条件，即

$$v_{FY} \in [70, 140] \quad (13)$$

5. **遮蔽有效性约束：**当导弹视野内真目标被完全遮蔽时才能将此时的时间微元计入总遮蔽时长，即

$$f(t) = \begin{cases} 0, & \max(d_n) > R \\ 1, & \max(d_n) \leq R \end{cases} \quad (14)$$

6. **烟幕干扰弹生效时长约束：**烟幕干扰弹在导弹击中假目标前的有效时间有限，即

$$t_{\text{stop}} = \min\left(20, \frac{L}{v_M} - t\right) \quad (15)$$

综上，本部分建立了使烟幕干扰弹有效遮蔽时长最大的优化模型

$$\begin{aligned} \max \quad & T(\theta, t_B, v_{FY}, t) = \int_0^{t_{\text{stop}}} f(t) dt \\ \text{s.t.} \quad & \begin{cases} \theta \in [0^\circ, 360^\circ) \\ 0 \leq t_B \leq \sqrt{\frac{2z_{FY}}{g}} \\ t_{FY} \geq 0 \\ v_{FY} \in [70, 140] \\ f(t) = \begin{cases} 0, & \max(d_n) > R \\ 1, & \max(d_n) \leq R \end{cases} \\ t_{\text{stop}} = \min\left(20, \frac{L}{v_M} - t\right) \end{cases} \end{aligned} \quad (16)$$

与 PSO 相比，本部分文采用的 APSO 在以下几个方面进行了针对性优化：

1. 收敛与多样性控制

在速度更新中引入 Clerc 收缩因子以增强算法稳定性，同时适当放宽速度上限以增加搜索范围；当种群进化陷入停滞时，通过围绕当前全局最优解的小幅高斯扰动实现局部重启，既保持了对最优解区域的深入探索，又避免过早收敛；

2. 边界处理机制

传统 PSO 在变量超出边界时通常采用反弹或随机扰动等方式处理，易导致粒子远离最优解所在的边界区域。本文方法则在粒子触及边界后对相应维度进行截断，并将法向速度分量置零，使粒子能够在边界附近沿切向稳定搜索，从而增强了对边界最优解的捕捉能力；

3. 目标函数评估

针对遮蔽时长计算对时间步长敏感的问题，本文在粗评估阶段引入“线性插零”的事件捕获策略，通过在函数值符号变化处进行插值修正，有效减小了步长误差；在最终结果阶段，再结合更小步长与二分法进行严格复评，确保了适应度函数计算的精确性；

4. 约束条件修复

在位置更新后引入依赖约束修复策略，并采用“可行优先”的比较准则，以避免不可行解对种群进化的干扰，从而保证搜索过程始终集中在可行域内。

5.2.2 问题二模型的求解

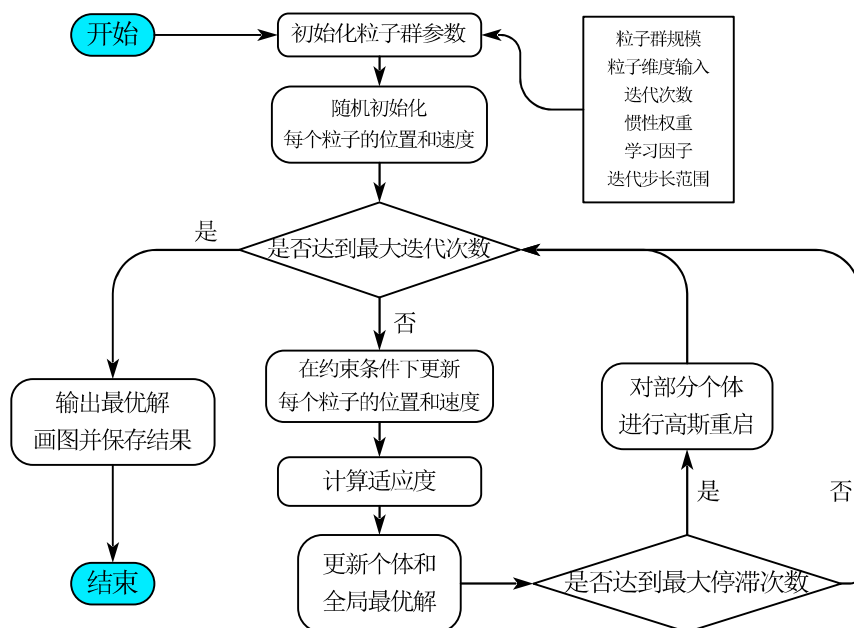


图6 改进的粒子群算法

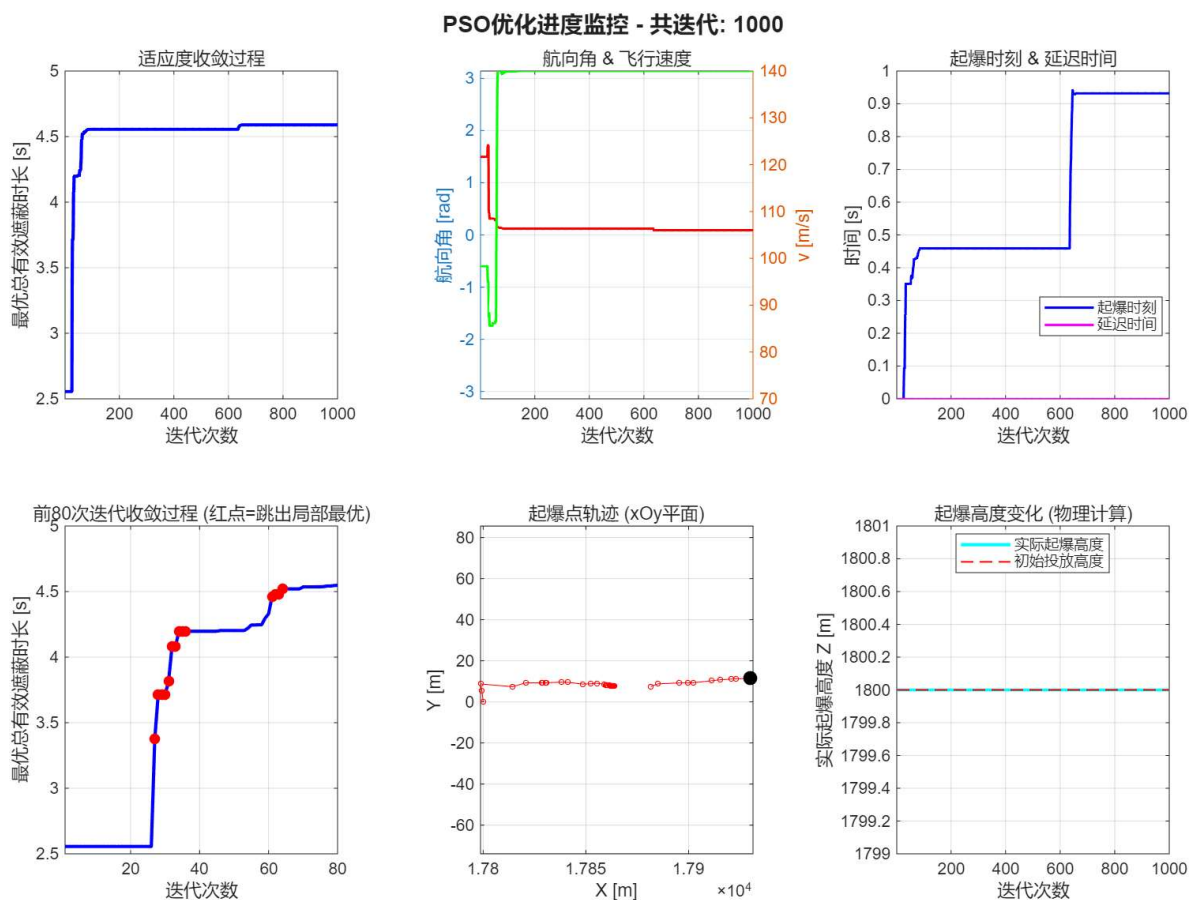


图7 更新迭代过程示意图

如 图 6 、 图 7 所示，在传统的粒子群算法^{[5][6][7]}中加入停滞时择机重启、动态调整惯性权重、对边界特殊处理的针对性优化之后，适应度收敛与跳出局部最优解都非常迅速，可以在很短的时间内收敛到全局最优解。

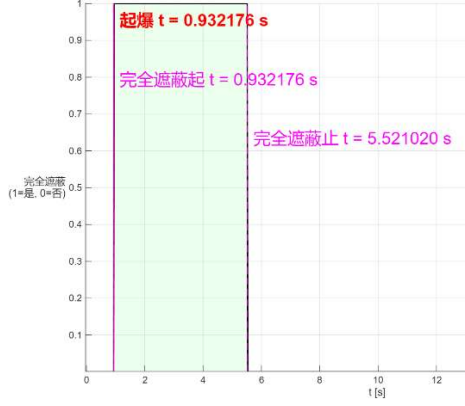


图 8 有效遮蔽时序图

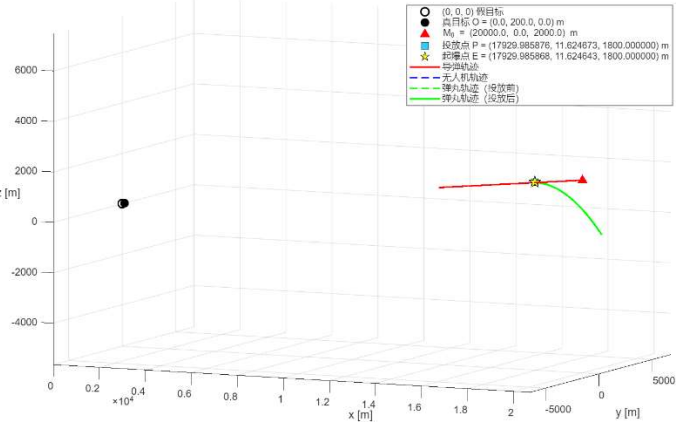


图 9 运动轨迹示意图

如 图 8 、 图 9 所示，模型最终计算结果为

$$\begin{aligned}
 \text{飞行方向} \quad \theta &= 16.0547^\circ \\
 \text{飞行速度} \quad v &= 140.00 \text{ m/s} \\
 \text{烟幕干扰弹投放点} \quad P &= (17929.985868, 11.624643, 1800.000000) \\
 \text{烟幕干扰弹起爆点} \quad E &= (17929.985868, 11.624643, 1800.000000) \\
 \text{总有效遮蔽时长} \quad T_{\text{cov}} &= 4.588844 \text{ s}
 \end{aligned} \tag{17}$$

5.3 问题三模型的建立与求解

5.3.1 问题三模型的建立

本部分在第二问的基础上，增加了烟幕干扰弹的投放数量以及投放间隔的限制。在此基础上，需要确定最佳的无人机飞行时长、速率、方向以及三枚烟幕干扰弹分别的投放与引爆时间，实现使烟幕干扰弹有效遮蔽时长最大的目标。

● 决策变量

欲使最终有效遮蔽时间最长，除了 θ 、 v_{FY} 以外，烟幕干扰弹数量的增多带来了额外变量：三枚烟幕干扰弹分别的投放时刻与引爆时间 t_j 、 t_{Bj} 。进行非线性优化是需寻找使有效遮蔽时间最长的 θ 、 v_{FY} 、 t_j 、 t_{Bj} 的具体数值解。因此，决策变量为 θ 、 v_{FY} 、 t_1 、 t_2 、 t_3 、 t_{B1} 、 t_{B2} 、 t_{B3} 。

● 目标函数

$$\max_x T(x) = \int_0^{t_{\text{stopj}}} 1_{\text{cov}}(t_j; x) dt \tag{18}$$

● 约束条件

1. 方位角约束：方位角取值范围不变，仍为 $[0^\circ, 360^\circ)$ ，即

$$\theta \in [0^\circ, 360^\circ) \quad (19)$$

2. 引爆时间约束：烟幕干扰弹引爆时间不能早于投放，不能晚于坠落，即

$$0 \leq t_{Bj} \leq \sqrt{\frac{2z_{FY}}{g}} \quad (20)$$

3. 飞行时间约束：无人机飞行时长非负，即

$$t_{FY} \geq 0 \quad (21)$$

4. 飞行速度约束：无人机的飞行速度有限制条件，即

$$v_{FY} \in [70, 140] \quad (22)$$

5. 遮蔽有效性约束：当导弹视野内真目标被完全遮蔽时才能将此时的时间微元计入总遮蔽时长，即

$$1_{\text{cov}}(t_j; x) = \begin{cases} 1, & \text{Covered}(t_j) \\ 0, & \text{otherwise} \end{cases} \quad (23)$$

6. 烟幕干扰弹生效时长约束：烟幕干扰弹在导弹击中假目标前的有效时间有限，即

$$t_{\text{stop}j} = \min\left(20, \frac{L}{v_M} - t_j\right) \quad (24)$$

7. 投放间隔约束：相邻两枚烟幕干扰弹投放间隔不能过小，即

$$t_{B3} - t_{B2} \geq 1, t_{B2} - t_{B1} \geq 1 \quad (25)$$

综上，本部分建立了使三枚烟幕干扰弹有效遮蔽时长综合最大的优化模型

$$\begin{aligned} \max_x \quad & T(x) = \int_0^{t_{\text{stop}j}} 1_{\text{cov}}(t_j; x) dt \\ \text{s.t.} \quad & \begin{cases} \theta \in [0^\circ, 360^\circ) \\ 0 \leq t_{Bj} \leq \sqrt{\frac{2z_{FY}}{g}} \\ t_{FY} \geq 0 \\ v_{FY} \in [70, 140] \\ 1_{\text{cov}}(t_j; x) = \begin{cases} 1, & \text{Covered}(t_j) \\ 0, & \text{otherwise} \end{cases} \\ t_{\text{stop}j} = \min\left(20, \frac{L}{v_M} - t_j\right) \end{cases} \end{cases} \quad (26)$$

其中，公式(23)中判断是否形成有效遮蔽的详细方法如下：

1. 采样集合及有效烟幕干扰弹集合定义

在时刻 t 时，定义采样集合 $Q(t)$ ，在多烟幕干扰弹协同遮蔽时为上表面圆盘和上表面圆在 xOy 面上的投影与圆柱的两个公切面，在单烟幕干扰弹完全遮蔽时则与先前相同，仍为上下两个外表面轮廓圆，定义当前有效烟幕干扰弹集合 $S(t)=\{j:t\in[t_{e,j}, t_{e,j}+20], z_j(t)\geq 0\}$ ；

令 $D_j(q,t)$ 为烟幕干扰弹中心 $C_j(t)$ 到线段 $M(t)q$ 的最短距离；

2. 多烟幕干扰弹协同（Union）

$$\text{Covered}(t) \iff \max_{q \in Q(t)} \min_{j \in S(t)} D_j(q,t) \leq r_s \quad (27)$$

3. 单烟幕干扰弹完全（single）

$$\text{Covered}(t) \iff \min_{j \in S(t)} \max_{q \in Q(t)} D_j(q,t) \leq r_s \quad (28)$$

在程序实现上，可以采用事件分段及自适应时间细化的方法近似计算该积分。在起爆、失效、落地等事件点分段，在各段内用有界变化率（Lipschitz 上界）进行安全判定及二分细化，得到覆盖区间并求和。

5.3.2 问题三模型的求解

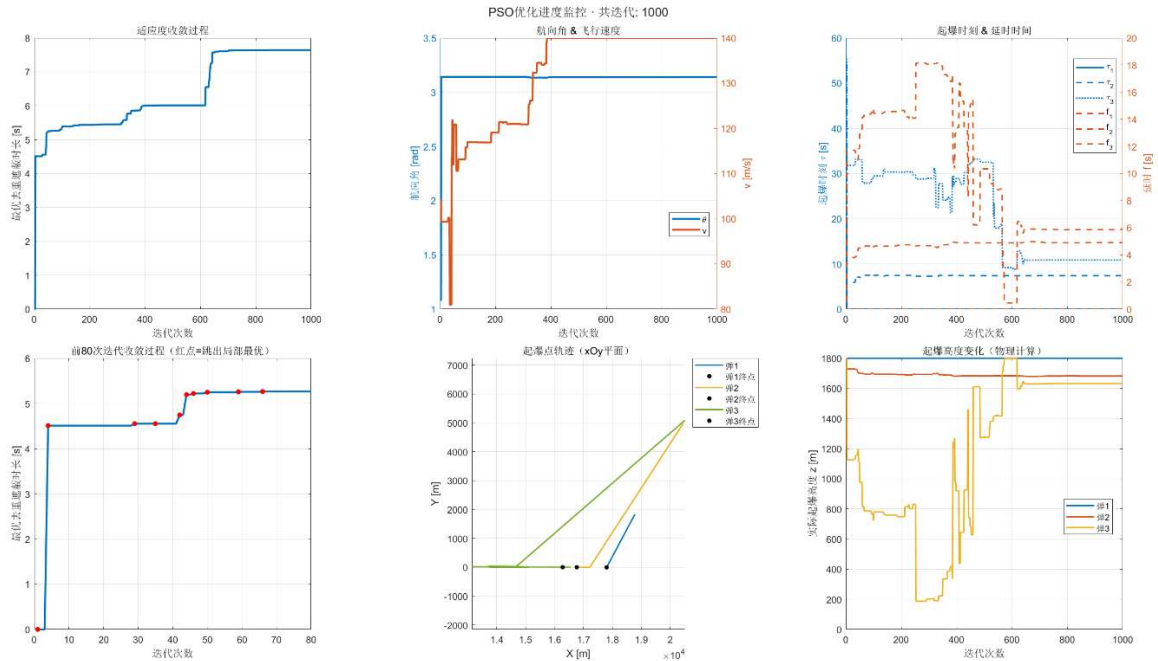


图 10 更新迭代过程示意图

如 图 10 所示, 在决策变量显著增多的时候, 事件分段及自适应时间细化的方式让目标函数收敛速度极为可观, 且有充足的能力跳出局部最优解。

最终模型计算的主要结果为

$$\begin{aligned}
 \text{第一枚烟幕干扰弹的有效遮蔽时间} \quad t_1 &= 2.556017 \text{ s} \\
 \text{第二枚烟幕干扰弹的有效遮蔽时间} \quad t_2 &= 3.456434 \text{ s} \\
 \text{第三枚烟幕干扰弹的有效遮蔽时间} \quad t_3 &= 1.632895 \text{ s} \\
 \text{总有效遮蔽时间} \quad T_{\text{cov}} &= 7.645346 \text{ s}
 \end{aligned} \tag{29}$$

完整结果详见 **result1.xlsx**。

5.4 问题四模型的建立与求解

5.4.1 问题四模型的建立

本部分在第二问的基础上, 增加了无人机的数量。在此基础上, 需要分别确定三架无人机的最佳飞行速率、方向以及烟幕干扰弹的最佳投放、引爆时间, 实现使烟幕干扰弹有效遮蔽时长最大的目标。

● 决策变量

欲使最终有效遮蔽时间最长, 对三架无人机、烟幕干扰弹均需要变量 θ_i 、 $v_{\text{FY}i}$ 、 t_i 、 $t_{\text{B}i}$ 。进行非线性优化是需寻找使有效遮蔽时间最长的量 θ_i 、 $v_{\text{FY}i}$ 、 t_i 、 $t_{\text{B}i}$ 的具体数值解。因此, 决策变量为 θ_1 、 $v_{\text{FY}1}$ 、 t_1 、 $t_{\text{B}1}$ 、 θ_2 、 $v_{\text{FY}2}$ 、 t_2 、 $t_{\text{B}2}$ 、 θ_3 、 $v_{\text{FY}3}$ 、 t_3 、 $t_{\text{B}3}$ 。

● 目标函数

$$\max_x T(x) = \int_0^{t_{\text{stop}i}} \text{cov}(t) dt \tag{30}$$

● 约束条件

1. 方位角约束: 方位角取值范围不变, 仍为 $[0^\circ, 360^\circ)$, 即

$$\theta \in [0^\circ, 360^\circ) \tag{31}$$

2. 引爆时间约束: 烟幕干扰弹引爆时间不能早于投放, 不能晚于坠落, 即

$$0 \leq t_{\text{B}i} \leq \sqrt{\frac{2z_{\text{FY}i}}{g}} \tag{32}$$

3. 飞行时间约束: 无人机飞行时长非负, 即

$$t_{\text{FY}i} \geq 0 \tag{33}$$

4. 飞行速度约束: 无人机的飞行速度有限制条件, 即

$$v_{FYi} \in [70, 140] \quad (34)$$

5. **遮蔽有效性约束：**当导弹视野内真目标被完全遮蔽时才能将此时的时间微元计入总遮蔽时长，即

$$\text{cov}(t) = \begin{cases} 1, & \text{Covered}_A(t_i) \vee \text{Covered}_B(t_i) \\ 0, & \text{otherwise} \end{cases} \quad (35)$$

6. **烟幕干扰弹生效时长约束：**烟幕干扰弹在导弹击中假目标前的有效时间有限，即

$$t_{\text{stop}i} = \min\left(20, \frac{L}{v_M} - t_i\right) \quad (36)$$

综上，本部分建立了使三枚烟幕干扰弹有效遮蔽时长综合最大的优化模型

$$\begin{aligned} \max_x T(x) &= \int_0^{t_{\text{stop}i}} \text{cov}(t) dt \\ s.t. &\begin{cases} \theta \in [0^\circ, 360^\circ] \\ 0 \leq t_{Bi} \leq \sqrt{\frac{2z_{FYi}}{g}} \\ t_{FYi} \geq 0 \\ v_{FYi} \in [70, 140] \\ \text{cov}(t) = \begin{cases} 1, & \text{Covered}_A(t_i) \vee \text{Covered}_B(t_i) \\ 0, & \text{otherwise} \end{cases} \\ t_{\text{stop}i} = \min\left(20, \frac{L}{v_M} - t_i\right) \end{cases} \end{aligned} \quad (37)$$

其中，公式(35)中判断是否形成有效遮蔽的详细方法如下：

1. 采样集合及有效烟幕干扰弹集合定义

在时刻 t 时，定义采样集合 $Q(t) = Q_{\text{top}} \cup Q_{\text{side}}^{\text{vis}}(t)$ ，其中

$$\begin{cases} \text{上表面整圆盘} & Q_{\text{top}} = \{(x, y, H_T) | (x-0)^2 + (y-200)^2 \leq R_T^2\} \\ \text{侧面可见侧} & (O_{xy} - C_{xy}) \cdot (A_{xy} - C_{xy}) \leq 0 \end{cases} \quad (38)$$

令 $D_i(q, t)$ 为烟幕干扰弹中心 $C_i(t)$ 到视线段 $\overline{M(t)q}$ 的最短距离，即

$$D_i(q, t) = \min_{u \in [0, 1]} \|C_i(t) - (M(t) + u(q - M(t)))\| \quad (39)$$

定义 $S(t)$ 为当前有效烟幕干扰弹集合，即

$$S(t) = \{i : t \in [t_{e,i}, t_{e,i} + 20], C_i(t)_z \geq 0\} \quad (40)$$

令 $D_j(q, t)$ 为烟幕干扰弹中心 $C_j(t)$ 到线段 $M(t)q$ 的最短距离；

2. 多烟幕干扰弹协同 (Union)

$$\text{Covered}_A(t) \iff \max_{q \in Q(t)} \min_{i \in S(t)} D_i(q, t) \leq r_s \quad (41)$$

3. 单烟幕干扰弹完全 (Single)

$$\text{Covered}_B(t) \iff \min_{i \in S(t)} \max_{q \in Q(t)} D_i(q, t) \leq r_s \quad (42)$$

在程序实现上，可以采用事件分段及自适应时间细化的方法逼近积分。在 $\{t_{e,i}, t_{e,i} + 20, t_{\text{stop}i}\}$ 等事件点分段，在各段内分别计算净空度 $\phi_A(t)$ 及 $\phi_B(t)$ ，做安全判定及二分，得到被遮蔽的时间子区间并求和，即可求得答案。其中净空度定义为

$$\begin{aligned} \phi_A(t) &= r_s - \max_{q \in Q(t)} \min_{i \in S(t)} D_i(q, t) \\ \phi_B(t) &= r_s - \min_{i \in S(t)} \max_{q \in Q(t)} D_i(q, t) \end{aligned} \quad (43)$$

5.4.2 问题四模型的求解

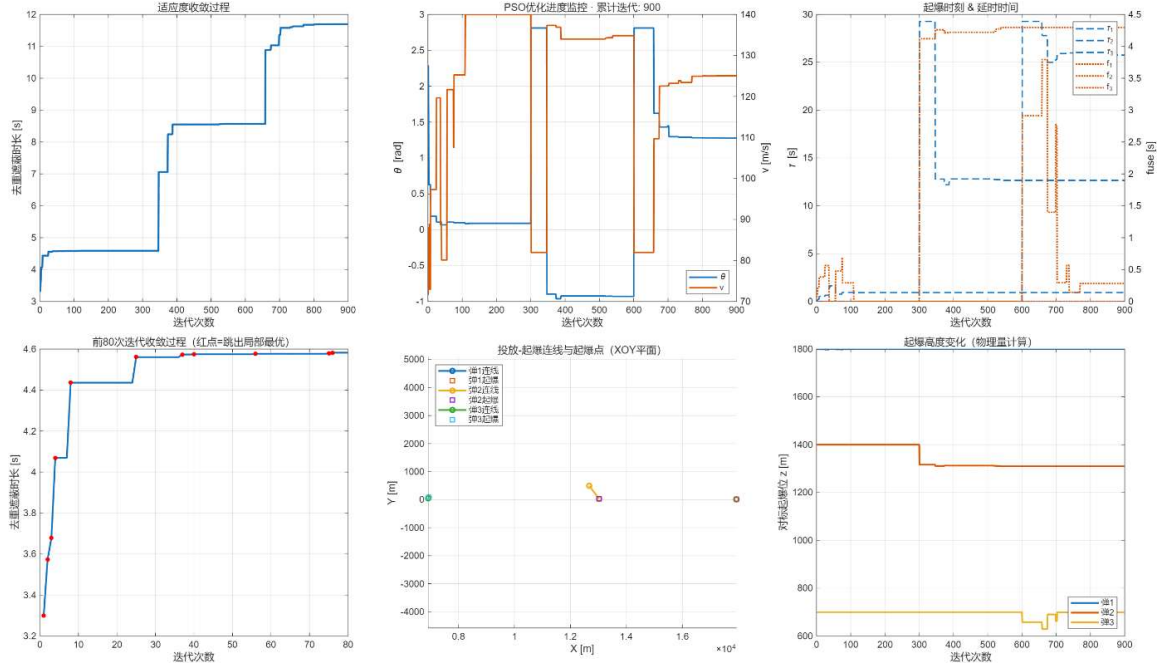


图 11 更新迭代过程示意图

如 图 11 所示，在决策变量显著增多的时候，“降维+贪心”的外层搭配多岛屿异质粒子群算法的内层让目标函数收敛速度极为可观，且有充足的能力跳出局部最优解。最终模型计算的主要结果为

$$\begin{aligned} t_1 &= 4.588845 \text{ s} \\ t_2 &= 3.979322 \text{ s} \\ t_3 &= 3.126111 \text{ s} \\ T_{\text{cov}} &= 11.694278 \text{ s} \end{aligned} \quad (44)$$

完整结果详见 **result2.xlsx**。

5.5 问题五模型的建立与求解

5.5.1 问题五模型的建立

本部分在第二问的基础上，增加了无人机、烟幕干扰弹和导弹的数量。在此基础上，需要分别确定三架无人机的最佳飞行速率、方向以及十五枚烟幕干扰弹的最佳投放、引爆时间，实现使烟幕干扰弹有效遮蔽时长最大的目标。

● 决策变量

欲使最终有效遮蔽时间最长，对三架无人机、十五枚烟幕干扰弹的安排需要变量 θ_i 、 v_{FYi} 、 t_{ij} 、 t_{Bij} 。进行非线性优化是需寻找使有效遮蔽时间最长的量 θ_i 、 v_{FYi} 、 t_{ij} 、 t_{Bij} 的具体数值解。因此，决策变量为 θ_i 、 v_{FYi} 、 t_{ij} 、 t_{Bij} 。

● 目标函数

$$\max F = (1 - \alpha) \sum_{j=1}^3 L_j + \alpha \sum_{j=1}^3 \ln(\varepsilon + L_j), \alpha \in [0, 1], \varepsilon > 0 \quad (45)$$

● 约束条件

1. 方位角约束：方位角取值范围不变，仍为 $[0^\circ, 360^\circ)$ ，即

$$\theta \in [0^\circ, 360^\circ) \quad (46)$$

2. 引爆时间约束：烟幕干扰弹引爆时间不能早于投放，不能晚于坠落，即

$$0 \leq t_{Bij} \leq \sqrt{\frac{2z_{FYi}}{g}} \quad (47)$$

3. 飞行时间约束：无人机飞行时长非负，即

$$t_{FYi} \geq 0 \quad (48)$$

4. 飞行速度约束：无人机的飞行速度有限制条件，即

$$v_{FYi} \in [70, 140] \quad (49)$$

5. 遮蔽有效性约束：当导弹视野内真目标被完全遮蔽时才能被计入总遮蔽时长，即

$$U_j = \bigcup_{k,b} I_{(k,b) \rightarrow j} \quad (50)$$

$$L_j = \mu(U_j) = \sum_{[a,b] \in U_j} (b - a)$$

6. 烟幕干扰弹生效时长约束：烟幕干扰弹在导弹击中假目标前的有效时间有限，即

$$t_{stopij} = \min\left(20, \frac{L}{v_M} - t_{ij}\right) \quad (51)$$

综上，本部分建立了使三枚烟幕干扰弹有效遮蔽时长综合最大的优化模型

$$\begin{aligned} \max F = & (1 - \alpha) \sum_{j=1}^3 L_j + \alpha \sum_{j=1}^3 \ln(\varepsilon + L_j), \alpha \in [0, 1], \varepsilon > 0 \\ s.t. \quad & \begin{cases} \theta \in [0^\circ, 360^\circ) \\ 0 \leq t_{Bij} \leq \sqrt{\frac{2z_{FYi}}{g}} \\ t_{FYi} \geq 0 \\ v_{FYi} \in [70, 140] \\ U_j = \bigcup_{k,b} I_{(k,b) \rightarrow j} \\ L_j = \mu(U_j) = \sum_{[a,b] \in U_j} (b - a) \\ t_{stopij} = \min\left(20, \frac{L}{v_M} - t_{ij}\right) \end{cases} \end{aligned} \quad (52)$$

5.5.2 问题五模型的求解

与问题四相比，问题五的主要区别在于参与主体和优化目标更复杂。第四问只涉及三架无人机、各投放一枚烟幕干扰弹，对单一导弹 M1 实施干扰，模型中决策变量主要是每架无人机的航向、速度以及对应弹药的释放时刻和引信延时，约束较为单一。而第五问扩展为五架无人机、每架最多可投放三枚烟幕干扰弹，并且同时要对三枚来袭导弹 M1、M2、M3 进行干扰。这样在决策变量上，不仅要考虑每架无人机的飞行参数和多枚弹的投放时序，还需引入弹药与导弹之间的指派变量；在约束条件上，需要保证单机投放间隔、投放总数限制，以及多目标导弹同时覆盖的时效约束。目标函数也从单导弹遮蔽时长最大化转变为三枚导弹遮蔽时长的综合优化，可采用公平性准则进行局部“贪心+降维”后结果的二次优化，利用时序权重因子，提升整体的遮蔽效果。

最终模型计算的主要结果为

$$T_{cov} = 22.195517 \text{ s} \quad (53)$$

完整结果详见 **result3.xlsx**。

六、模型优缺点评价

6.1 模型的优点

模型效率高：本研究跳出了传统表面取点的局限，转而采用严谨的数学、几何证明简化模型，例如将三维表面简化为二维线条，并进一步简化为一位特征点，又如将视野遮蔽问题简化为。

6.2 模型的缺点

本研究因计算机运算时间有限，难以做到关于所有决策变量的动态规划，出于运行时间的考虑，结合一定的数学论证，在恰当的地方转而使用贪心算法以提高效率，可能无法达到全局最优解，但仍可以以高效率得出较为优良的结果。

参考文献

- [1] 顾文慧,高红杰.烟幕投放控制分析与设计[J].光电技术应用,2014,29(02):87-89.
- [2] ChatGPT,ChatGPT 5,OpenAI,2025-09-04
- [3] ChatGPT,ChatGPT 5,OpenAI,2025-09-05
- [4] ChatGPT,ChatGPT 5,OpenAI,2025-09-05
- [5] ChatGPT,ChatGPT 5,OpenAI,2025-09-06
- [6] 张利彪,周春光,马铭,等.基于粒子群算法求解多目标优化问题[J].计算机研究与发展,2004,(07):1286-1291.
- [7] 刘建华.粒子群算法的基本理论及其改进研究[D].中南大学,2009.

附录

附录 1

介绍：支撑材料的文件列表

better.m
cloud_center.m
dist_point_to_segments.m
explosion_point.m
fit_union.m
fitness_c.m
fitness_x.m
fitness1_marginal.asv
fitness1_marginal.m
gfun_t.m
interp_zero_intervals.m
intervals_one.m
intervals_one_c.m
intervals_one_x.m
intervals_x.m
island_params.m
missile_pos.m
refine_edge.m
result1.xlsx
result2.xlsx
result3.xlsx
sample_disks.m
solve_one_uav_islands.asv
solve_one_uav_islands.m
t_1.m
t_2.m
t_3.m
t_4.asv
t_4.m
t_5.asv
t_5.m
total_len.m
union_all.m

附录 2

包含重要的三个重要命题的证明，支撑了代码的优化

命题一

可以证得，如图一，从一个点光源对球体作投影，投到平面上的阴影（轮廓）是椭圆。下面给出完整证明。

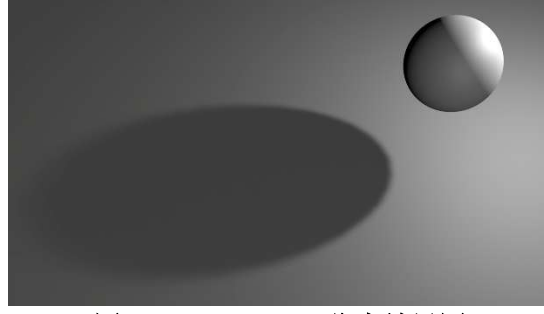


图 12 Cinema 4D 仿真效果图

设球心为 C ，半径为 R ；点光源为 S ；投影平面为 Π ，任取平面上坐标点 P ；令向量 $\mathbf{u} = P - S$ ， $\mathbf{w} = C - S$ ；点 P 在阴影边界 $\Leftrightarrow \text{dist}(C, SP) = R$ ；因此边界条件为

$$\text{dist}(C, SP) = \frac{\|\mathbf{u} \times \mathbf{w}\|}{\|\mathbf{u}\|} = R \quad (54)$$

两边分别平方得到

$$\|\mathbf{u} \times \mathbf{w}\|^2 = R^2 \|\mathbf{u}\|^2 \quad (55)$$

把 $\mathbf{u} = P - S$ 展开，将叉积与范数展开，可以得到关于平面坐标的二次型方程：

$$A\mathbf{u}^2 + B\mathbf{uv} + C\mathbf{v}^2 + D\mathbf{u} + E\mathbf{v} + F = 0$$

$$\begin{cases} A = (w_y^2 + w_z^2 - R^2) \\ B = -2w_x w_y uv \\ C = (w_x^2 + w_z^2 - R^2) \\ D = 2(x_s(R^2 - w_y^2 - w_z^2) + w_x(w_y y_s + w_z z_s)) \\ E = 2(y_s(R^2 - w_x^2 - w_z^2) + w_y(w_x x_s + w_z z_s)) \\ F = (x_s^2 + y_s^2 + z_s^2)(x_c^2 + y_c^2 + z_c^2 - R^2) - (x_s x_c + y_s y_c + z_s z_c)^2 \end{cases} \quad (56)$$

根据二次曲线方程可以判断，在当前情形下，阴影轮廓一定为椭圆；证毕。

命题二

可以证得，如图二，从点光源把一个平面内的圆投影到与该圆平行的另一个平面上，投影轮廓仍然是圆。下面给出完整证明。

记点光源为 S ；从 S 到两平面的公共法线在圆所在面与投影所在面上的交点分别记为 A 和 A' ；对于原平面上一点 P ，连线 SP 与投影平面的交点为 Q ；

由于顶角相等， $AP \parallel A'Q$ ，易得 $\triangle SAP \sim \triangle SA'Q$ ；

因此对任意 P 满足 $\frac{A'Q}{AP} = \lambda$ ，其中 λ 为常数；

原圆是满足 $AP = R$ 的点集，故所有 $A'Q = \lambda AP$ ；所以投影得到的点集满足 $A'Q = \lambda R$ ，即是以 A' 为中心、半径为 λR 的圆；

证毕。

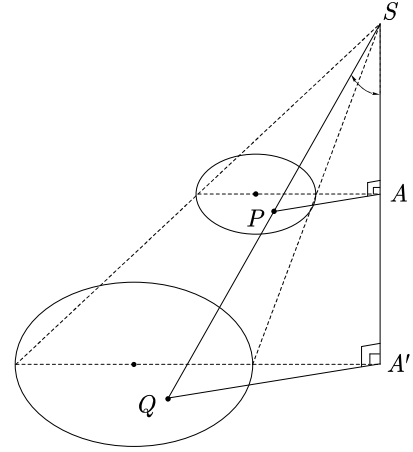


图 13 点光源投影示意图

命题三

可以证得，如果一个两个相离的圆均在同一椭圆内，则这由两个圆以及二者的两条外公切线包络形成的形状也完全在该椭圆内。下面给出完整证明。

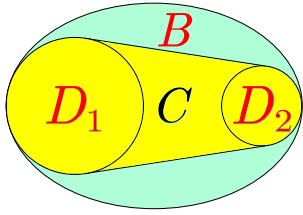


图 14 几何关系示意图

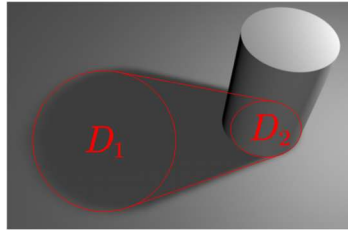


图 15 圆柱投影示意图

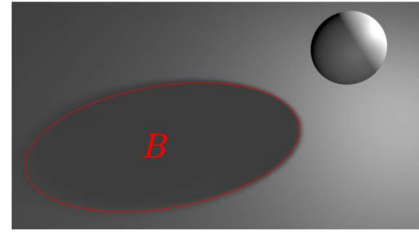


图 16 球投影示意图

证明：

由于椭圆是圆的仿射像，故该仿射像保持了圆的凸性，可得图形 B 为凸集；

将圆柱上表面轮廓圆在 xOy 面上的投影圆记为 D_1 ，半径为 r_1 ；下表面轮廓圆记为 D_2 ，半径为 r_2 ；

则存在关系 $D_1 \subseteq C$ 与 $D_2 \subseteq C$ ，因此 $D_1 \cup D_2 \subseteq B$ ；

记 $C = \bigcup_{t \in [0, 1]} D((1-t)c_1 + tc_2, (1-t)r_1 + tr_2)$ ；

- 证明 $\text{conv}(D_1 \cup D_2) \subseteq C$

取凸包内任意一点 x ，按照凸包定义，其可以写为

$$x = (1-t)x_1 + tx_2, x_1 \in D_1, x_2 \in D_2, t \in [0, 1] \quad (57)$$

把 x 写为中心加偏移： $x_i = c_i + u_i, \|u_i\| \leq r_i$ 。代回得

$$x = (1-t)c_1 + tc_2 + (1-t)u_1 + tu_2 \quad (58)$$

由三角不等式得

$$\|(1-t)u_1 + tu_2\| \leq (1-t)r_1 + tr_2 \quad (59)$$

所以 x 落在圆盘 $D((1-t)c_1 + tc_2, (1-t)r_1 + tr_2)$ 内；

这正是 C 中的一员，所以 $x \in C$ ；

因此， $\text{conv}(D_1 \cup D_2) \subseteq C$ 。

- 证明 $C \subseteq \text{conv}(D_1 \cup D_2)$

取 C 内任意一点 y ，按照定义，存在 $t \in [0, 1]$ 与向量 $\mathbf{v}$ 使得

$$y = (1-t)c_1 + tc_2 + \mathbf{v}, \|\mathbf{v}\| \leq (1-t)r_1 + tr_2 \quad (60)$$

$$\text{令方向单位向量 } \mathbf{u} = \begin{cases} \mathbf{v}/((1-t)r_1 + tr_2) & (\mathbf{v} \neq \mathbf{0}) \\ \mathbf{0} & (\mathbf{v} = \mathbf{0}) \end{cases};$$

定义两点 $p_1 = c_1 + r_1 \mathbf{u} \in D_1$, $p_2 = c_2 + r_2 \mathbf{u} \in D_2$;

则有

$$y = (1-t)p_1 + tp_2 \quad (61)$$

也就是 y 是 $D_1 \cup D_2$ 中两点的凸组合，所以 $y \in \text{conv}(D_1 \cup D_2)$ ；

因此， $C \subseteq \text{conv}(D_1 \cup D_2)$ 。

由上述二点可得

$$\text{conv}(D_1 \cup D_2) = D = C \quad (62)$$

由于任意凸集若包含一个集合，就包含该集合的凸包；

因此

$$\text{conv}(D_1 \cup D_2) = C \subseteq B \quad (63)$$

即只要图形 C 可以完全包住上表面的投影圆面 D_1 及下表面圆面 D_2 ，其就一定能把整个由两个圆以及二者的两条外公切线包络形成的形状完全包住；
证毕。

结论：只要圆柱的上下表面圆周均未出现在导弹视野中，即可保证整个圆柱都无法观察到，此时则可以认为真目标被有效遮蔽。

附录 3

better.m

```
function B = better(f1,feas1, f2,feas2)
    if feas1 && ~feas2, B=true;
    elseif ~feas1 && feas2, B=false;
    else, B=(f1>f2);
    end
end
```

附录 3

cloud_center.m

```
function c = cloud_center(t, e, vs, tau)
    c = [e(1), e(2), e(3) - vs*(t - tau)];
end
```

附录 3

dist_point_to_segments.m

```
function d = dist_point_to_segments(c, a, B)
    BA = B - a;
    denom = sum(BA.*BA,2);
    denom = max(denom,1e-12);
    s = sum((c - a).*BA,2)./denom;
    s=max(0,min(1,s));
    P = a + s.*BA;
    d = vecnorm(P - c, 2, 2);
end
```

附录 3

explosion_point.m

```
function [xe,ye,ze] = explosion_point(fy0, theta, v, tau, fuse, g)
    xe = fy0(1) + v*tau*cos(theta);
    ye = fy0(2) + v*tau*sin(theta);
    ze = fy0(3) - 0.5*g*fuse^2;
end
```

附录 3

fitness1_marginal.m

```
function [fit_marg, feasible] = fitness1_marginal(x, FYk, Q, dt, M0, u_m, vd,
vg, rg, tg, T_hit, fmax_k, U_prev)
    feasible=true;
    fit_marg=-1e9;
    theta=x(1);
    v=x(2);
    t_b=x(3);
    t_l=x(4);
    if v<70 || v>140 || t_b<0 || t_b>T_hit || t_l<0 || t_l>min(t_b,fmax_k)
        feasible=false;
        return;
    end
    ez = FYk(3) - 0.5*9.8*t_l^2;
    if ez<0, feasible=false;
```



```

        return;
    end

    iv = intervals_one_c(x, FYk, Q, dt, M0, u_m, vd, vg, rg, tg, T_hit);
    Unew = union_all([U_prev; iv]);
    fit_marg = total_len(Unew) - total_len(U_prev);
end

```

附录 3

fitness_c.m

```

function [T_C, feasible] = fitness_c(x, FY, m0, u_m, vm, vs, rs, life, Q, dt,
g, fmax)
    theta=x(1); v=x(2); t_b=x(3); t_l=x(4);
    feasible=true;
    if v<70 || v>140 || t_b<0 || t_b>norm(m0)/vm || t_l<0, feasible=false; end
    if t_l>t_b || t_l>fmax, feasible=false; end
    ez = FY(3) - 0.5*g*t_l^2; if ez<0, feasible=false; end
    if ~feasible, T_C=-1e9; return; end

    T_end = min(t_b+life, norm(m0)/vm);
    if T_end<=t_b, T_C=0; return; end

    [xe,ye,~] = explosion_point(FY, theta, v, t_b, t_l, g);
    ts = t_b:dt:T_end;
    if ts(end)<T_end, ts=[ts, T_end]; end % 保证末端包含
    vals = zeros(numel(ts),1);

    for k=1:numel(ts)
        t=ts(k);
        c = [xe, ye, ez - vs*(t - t_b)];
        m = missile_pos(t, m0, u_m, vm);
        vals(k) = gfun_t(c,m,Q,rs);
    end
    % 线性插零近似遮蔽段
    % mask = vals<=0;
    T_C = 0; in=false; t_on=NaN;

    for k=2:numel(ts)
        v1=vals(k-1); v2=vals(k); t1=ts(k-1); t2=ts(k);
        if ~in && v1>0 && v2<=0 % 下降穿越
            t_on = t1 + (0 - v1)*(t2 - t1)/(v2 - v1);
            in = true;
        elseif ~in && v1<=0 && v2<=0 && k==2 % 起点就在覆盖内

```

```

        t_on = t1; in=true;
    end
    if in && v1<=0 && v2>0 % 上升穿越
        t_off = t1 + (0 - v1)*(t2 - t1)/(v2 - v1);
        T_C = T_C + max(0, t_off - t_on);
        in=false;
    elseif in && k==numel(ts) % 末尾仍在覆盖内
        t_off = t2;
        T_C = T_C + max(0, t_off - t_on);
        in=false;
    end
end
end
end

```

附录 3

fitness_x.m

```

function [T_C, covered, out] = fitness_x(x, FY, m0, u_m, vm, vs, rs, life, Q,
dt, g)
    theta=x(1);
    v=x(2);
    t_b=x(3);
    t_l=x(4);
    [xe,ye,ez] = explosion_point(FY, theta, v, t_b, t_l, g);
    T_end = min(t_b+life, norm(m0)/vm);
    out.tStart=t_b;
    out.tEnd=T_end;
    out.intervals=[];
    out.durations=[];
    out.T_C=0;
    if T_end<=t_b || ez<0, T_C=0;
        covered=[];
        return;
    end
    gfun = @(t) gfun_t([xe,ye, ez - vs*(t - t_b)], missile_pos(t,m0,u_m,vm),
Q, rs);
    ts = t_b:dt:T_end;
    if ts(end)<T_end, ts=[ts,T_end]; end
    vals = arrayfun(gfun, ts);
    covered = vals<=0;

    intervals=[];
    if covered(1)
        t_on = refine_edge(@(tt) gfun(tt)<=0, t_b, min(t_b+dt, T_end), true);
    end
end

```

```

else
    t_on=[];
end
for k=2:numel(ts)
    if ~covered(k-1) && covered(k)
        t_on = refine_edge(@(tt) gfun(tt)<=0, ts(k-1), ts(k), true);
    elseif covered(k-1) && ~covered(k)
        t_off= refine_edge(@(tt) gfun(tt)<=0, ts(k-1), ts(k), false);
        intervals=[intervals; [t_on, t_off]]; %#ok<AGROW>
        t_on=[];
    end
end
if ~isempty(t_on) && covered(end)
    t_off = refine_edge(@(tt) gfun(tt)<=0, max(T_end-dt,t_b), T_end,
false);
    intervals=[intervals; [t_on, t_off]];
end
if isempty(intervals)
    durations = [];
    T_C = 0;
else
    durations = max(0, intervals(:,2)-intervals(:,1));
    T_C = sum(durations);
end
out.intervals=intervals; out.durations=durations; out.T_C=T_C;
end

```

附录 3

fit_union.m

```

function [fitval, feasible] = fit_union(x, fy0, m0, u_m, vm, vs, rs, life, Q,
dt, g, fmax, T_hit)
    theta = x(1); v = x(2);
    tau = [x(3), x(5), x(7)];
    fus = [x(4), x(6), x(8)];
    feasible = true;
    if ~(v>=70 && v<=140), feasible=false; end
    if any(tau<0) || any(tau>T_hit) || any(fus<0), feasible=false; end
    fus = min(fus, fmax);

    for j=1:3
        if fus(j) > tau(j), feasible=false; end
        ez = fy0(3) - 0.5*g*fus(j)^2;
        if ez < 0, feasible=false; end
    end

```

```

end
if ~feasible, fitval=-1e9; return; end
rel = tau - fus;
[rel_s, idx] = sort(rel);
tau = tau(idx); fus = fus(idx); rel = rel_s;
if rel(1)<0 || rel(2)<rel(1)+1 || rel(3)<rel(2)+1
    feasible=false; fitval=-1e9; return;
end

intervals = cell(3,1);
for j=1:3
    intervals{j} = intervals_one(theta, v, tau(j), fus(j), fy0, m0, u_m,
vm, vs, rs, life, Q, dt, g, T_hit);
end
U = union_all(intervals);
fitval = total_len(U);
end

```

附录 3

gfun_t.m

```

function val = gfun_t(c, m, Q, rs)
    if c(3) < 0
        val = +inf; % 地面以下视为不可能覆盖
        return;
    end
    val = max(dist_point_to_segments(c,m,Q)) - rs;
end

```

附录 3

interp_zero_intervals.m

```

function iv = interp_zero_intervals(ts, vals)
    iv=zeros(0,2); in=false; t_on=NaN;
    for k=2:numel(ts)
        v1=vals(k-1); v2=vals(k); t1=ts(k-1); t2=ts(k);
        if ~in && v1>0 && v2<=0, t_on = t1 + (0 - v1)*(t2 - t1)/(v2 - v1);
in=true;
        elseif ~in && v1<=0 && v2<=0 && k==2, t_on=t1; in=true;
        end
        if in && v1<=0 && v2>0
            t_off = t1 + (0 - v1)*(t2 - t1)/(v2 - v1);
            iv = [iv; [t_on, t_off]]; in=false;
        elseif in && k==numel(ts)

```

```

        iv = [iv; [t1, t2]]; in=false;
    end
end
end

```

附录 3

intervals_one_c.m

```

function iv = intervals_one_coarse(x, FYk, Q, dt, m0, u_m, vm, vs, rs, life,
T_hit)
    theta=x(1); v=x(2); tau=x(3); fuse=x(4);
    T_end = min(tau+life, T_hit); iv=zeros(0,2);
    if T_end<=tau, return; end
    xe = FYk(1)+v*tau*cos(theta);
    ye = FYk(2)+v*tau*sin(theta);
    ze = FYk(3)-0.5*9.8*fuse^2;
    ts = tau:dt:T_end; if ts(end)<T_end, ts=[ts,T_end]; end
    vals = zeros(numel(ts),1);
    for k=1:numel(ts)
        t=ts(k);
        c=[xe,ye, ze - vs*(t - tau)];
        m = missile_pos(t, m0, u_m, vm);
        vals(k) = gfun_t(c,m,Q,rs);
    end
    iv = interp_zero_intervals(ts, vals);
end

```

附录 3

intervals_one.m

```

function iv = intervals_one(theta, v, tau, fuse, fy0, m0, u_m, vm, vs, rs,
life, Q, dt, g, T_hit)
% 单弹在 [tau, min(tau+life,T_hit)] 的遮蔽区间（线性插零 + 二分边界近似）
    ez = fy0(3) - 0.5*g*fuse^2;
    if ez < 0, iv = zeros(0,2); return; end
    T_end = min(tau+life, T_hit);
    if T_end <= tau, iv = zeros(0,2); return; end

% 起爆点水平
    xe = fy0(1) + v*tau*cos(theta);
    ye = fy0(2) + v*tau*sin(theta);

    ts = tau:dt:T_end;
    if ts(end) < T_end, ts = [ts, T_end]; end

```

```

vals = zeros(numel(ts),1);
for k=1:numel(ts)
    t = ts(k);
    c = [xe, ye, ez - vs*(t - tau)];
    m = m0 + vm*(u_m*t);
    vals(k) = max(dist_point_to_segments(c, m, Q)) - rs;
end
% 线性插零求粗区间
iv = zeros(0,2); in=false; t_on=NaN;
for k=2:numel(ts)
    v1=vals(k-1); v2=vals(k); t1=ts(k-1); t2=ts(k);
    if ~in && v1>0 && v2<=0, t_on = t1 + (0 - v1)*(t2 - t1)/(v2 - v1);
in=true;
    elseif ~in && v1<=0 && v2<=0 && k==2, t_on=t1; in=true;
    end
    if in && v1<=0 && v2>0
        t_off = t1 + (0 - v1)*(t2 - t1)/(v2 - v1);
        iv(end+1,:) = [t_on, t_off]; %#ok<AGROW>
        in=false;
    elseif in && k==numel(ts)
        iv(end+1,:) = [t_on, t2]; %#ok<AGROW>
        in=false;
    end
end
end
end

```

附录 3

intervals_one_x.m

```

function [iv, dur] = intervals_one_x(x, FYk, Q, dt, m0, u_m, vm, vs, rs, tg,
T_hit, g)
    theta=x(1); v=x(2); t_b=x(3); t_l=x(4);
    iv=zeros(0,2); dur=zeros(0,1);
    ez = FYk(3) - 0.5*g*t_l^2; if ez<0, return; end
    T_end = min(t_b+tg, T_hit); if T_end<=t_b, return; end
    xe = FYk(1)+v*t_b*cos(theta);
    ye = FYk(2)+v*t_b*sin(theta);
    gfun = @(t) (max(dist_point_to_segments([xe,ye, ez - vs*(t - t_b)],
missile_pos(t,m0,u_m,vm), Q)) - rs);

    ts = t_b:dt:T_end; if ts(end)<T_end, ts=[ts,T_end]; end
    vals = arrayfun(gfun, ts); covered = vals<=0;

    if covered(1)

```

```

        t_on = refine_edge(@(tt) gfun(tt)<=0, t_b, min(t_b+dt, T_end), true);
    else
        t_on = NaN;
    end
    for k=2:numel(ts)
        if ~covered(k-1) && covered(k)
            t_on = refine_edge(@(tt) gfun(tt)<=0, ts(k-1), ts(k), true);
        elseif covered(k-1) && ~covered(k)
            t_off= refine_edge(@(tt) gfun(tt)<=0, ts(k-1), ts(k), false);
            iv = [iv; [t_on, t_off]];
        end
    end
    if ~isnan(t_on) && covered(end)
        t_off= refine_edge(@(tt) gfun(tt)<=0, max(T_end-dt,t_b), T_end, false);
        iv=[iv; [t_on, t_off]];
    end
    if ~isempty(iv), dur=max(0,iv(:,2)-iv(:,1)); end
end

```

附录 3

intervals_x.m

```

function [iv, T_C] = intervals_x(theta, v, t_b, fuse, fy0, m0, u_m, vm, vg,
rs, tg, Q, dt, g, T_hit)
[xe,ye,ez] = explosion_point(fy0, theta, v, t_b, fuse, g);
if ez < 0, iv=zeros(0,2); T_C=0; return; end
T_end = min(t_b+tg, T_hit);
if T_end <= t_b, iv=zeros(0,2); T_C=0; return; end

gfun = @(t) ( max(dist_point_to_segments(cloud_center(t, [xe, ye, ez], vg,
t_b), missile_pos(t, m0, u_m, vm), Q)) - rs );

ts = t_b:dt:T_end; if ts(end)<T_end, ts=[ts, T_end]; end
vals = arrayfun(gfun, ts); covered = vals<=0;

iv = zeros(0,2);
if covered(1)
    t_on = refine_edge(@(tt) gfun(tt)<=0, t_b, min(t_b+dt, T_end), true);
else
    t_on = [];
end
for k=2:numel(ts)
    if ~covered(k-1) && covered(k)
        t_on = refine_edge(@(tt) gfun(tt)<=0, ts(k-1), ts(k), true);
    end
end

```

```

elseif covered(k-1) && ~covered(k)
    t_off= refine_edge(@(tt) gfun(tt)<=0, ts(k-1), ts(k), false);
    iv(end+1,:) = [t_on, t_off];
    t_on = [];
end
end
if ~isempty(t_on) && covered(end)
    t_off = refine_edge(@(tt) gfun(tt)<=0, max(T_end-dt,t_b), T_end,
false);
    iv(end+1,:) = [t_on, t_off];
end
T_C = total_len(iv);
end

```

附录 3

island_params.m

```

function P = island_params(wmax,wmin,c1,c2,vmax_scale)
    P.w_max=wmax; P.w_min=wmin; P.c1=c1; P.c2=c2; P.vmax_scale=vmax_scale;
end

```

附录 3

missile_pos.m

```

function m = missile_pos(t, m0, u_m, vm)
    m = m0 + vm .* (u_m .* t);
end

```

附录 3

refine_edge.m

```

function t_edge = refine_edge(isCovered, ta, tb, rising)
    % 在 [ta,tb] 内二分求覆盖边界（保 6 位小数）
    % rising=true : 从不覆盖->覆盖 的交点
    % rising=false : 从覆盖->不覆盖 的交点
    fa = isCovered(ta);
    fb = isCovered(tb);
    % 若端点状态不符预期，尝试纠正（容错）
    if rising && ~(~fa && fb)
        % 扩展一次小范围
        mid = (ta+tb)/2;
        fa = isCovered(ta); fm = isCovered(mid); fb = isCovered(tb);
        if ~fa && fm, tb = mid; fb = fm; end
    elseif ~rising && ~(fa && ~fb)

```



```

        mid = (ta+tb)/2;
        fa = isCovered(ta); fm = isCovered(mid); fb = isCovered(tb);
        if fm && ~fb, ta = mid; fa = fm; end
    end
    % 标准二分
    for iter = 1:80
        tm = (ta + tb)/2;
        fm = isCovered(tm);
        if rising
            if fm, tb = tm; else, ta = tm; end
        else
            if fm, ta = tm; else, tb = tm; end
        end
        if (tb - ta) <= 1e-7, break; end % 时间精度 ~1e-7 s, 输出保留 6 位
    end
    t_edge = round((ta+tb)/2, 6); % 最终四舍五入到小数点后 6 位
end

```

附录 3

sample_disks.m

```

function Q = sample_disks(CY, RT, H, Nl, Nr)
    thetas = linspace(0,2*pi,Nl+1); thetas(end)=[];
    radii = linspace(0,RT,Nr);
    Q = [];
    for zlevel=[0,H]
        [RR,TT]=meshgrid(radii,thetas);
        XX = CY(1)+RR(:).*cos(TT(:));
        YY = CY(2)+RR(:).*sin(TT(:));
        ZZ = zlevel*ones(numel(XX),1);
        Q = [Q; [XX,YY,ZZ]]; %#ok<AGROW>
    end
    Q = unique(round(Q,10),'rows');
end

```

附录 3

solve_one_uav_islands.m

```

function xbest = solve_one_uav_islands(FYk, fmax_k, Qc, dt, m0, u_m, vm, vs,
rs, life, T_hit, U_prev)
    D = 4;
    K = 3;
    pop_island = 50;
    iterMax = 300;

```

```

stallMax = 25;
migPeriod = 20;
lb = [-pi, 70, 0, 0];
ub = [pi, 140, T_hit, fmax_k];
base_range = [2*pi, 70, T_hit, fmax_k];
isParams(1) = island_params(0.95,0.60, 1.6,1.4, 0.60);
isParams(2) = island_params(0.90,0.5, 1.8,1.8, 0.40);
isParams(3) = island_params(0.80,0.30, 1.4,2.2, 0.30);

islands =
repmat(struct('X',[],'V',[],'pbest',[],'pbest_fit',[],'pbest_feas',[], ...
              'gbest',[],'gbest_fit',-
inf,'gbest_feas',false,'bestHist',[], ...
              'stall',0,'params',[],'vmax',[]), K, 1);

for s=1:K
    rng(12588+s); prm=isParams(s);
    islands(s).params=prm;
    islands(s).vmax=prm.vmax_scale*base_range;
    X=zeros(pop_island,D);
    V=zeros(pop_island,D);
    for i=1:pop_island
        X(i,1) = (rand*2-1)*pi;
        X(i,2) = 70 + 70*rand;
        X(i,3) = T_hit*rand;
        X(i,4) = rand * min(X(i,3), fmax_k);
    end
    for j=1:D
        V(:,j)=(rand(pop_island,1)*2-1).*islands(s).vmax(j);
    end
    islands(s).X=X; islands(s).V=V; islands(s).pbest=X;
    pf=zeros(pop_island,1); feas=false(pop_island,1);
    for i=1:pop_island
        [pf(i),feas(i)] = fitness1_marginal(X(i,:), FYk, Qc, dt, m0, u_m,
vm, vs, rs, life, T_hit, fmax_k, U_prev);
    end
    islands(s).pbest_fit=pf;
    islands(s).pbest_feas=feas;
    [islands(s).gbest_fit,ib]=max(pf);
    islands(s).gbest=X(ib,:);
    islands(s).gbest_feas=feas(ib);
    islands(s).bestHist=zeros(iterMax,1);
    islands(s).stall=0;
end

```

```

for it=1:iterMax
    for s=1:K
        prm=islands(s).params;
        vmax=islands(s).vmax;
        w = prm.w_max - (prm.w_max - prm.w_min)*(it-1)/(iterMax-1);
        phi=prm.c1+prm.c2; chi=2/(phi-2+sqrt(max(phi^2-4*phi,1e-12)));

        X=islands(s).X;
        V=islands(s).V;
        pbest=islands(s).pbest;
        pf=islands(s).pbest_fit;
        feas=islands(s).pbest_feas;
        gbest=islands(s).gbest;
        gfit=islands(s).gbest_fit;
        gfeas=islands(s).gbest_feas;

        r1=rand(pop_island,D);
        r2=rand(pop_island,D);
        V = chi*( w*V + prm.c1*r1.*(pbest-X) +
prm.c2*r2.*( repmat(gbest,pop_island,1)-X) );
        for j=1:D
            V(:,j)=max(-vmax(j),min(vmax(j),V(:,j)));
        end

        X = X + V;
        for i=1:pop_island
            xi=X(i,:); xi=max(lb,min(ub,xi));
            tau=xi(3); fk=xi(4);
            tau=min(max(tau,0),T_hit); fk=min(max(fk,0),min(tau,fmax_k));
            xi(3)=tau; xi(4)=fk; xi(2)=min(max(xi(2),70),140);
            X(i,:)=xi;
        end
        touched=(X<=repmat(lb,pop_island,1))|(X>=repmat(ub,pop_island,1));
        V(touched)=0;

        fit=zeros(pop_island,1); fea=false(pop_island,1);
        for i=1:pop_island
            [fit(i),fea(i)] = fitness1_marginal(X(i,:), FYk, Qc, dt, m0,
u_m, vm, vs, rs, life, T_hit, fmax_k, U_prev);
        end

        for i=1:pop_island
            if better(fit(i),fea(i), pf(i),feas(i))
                pbest(i,:)=X(i,:); pf(i)=fit(i); feas(i)=fea(i);

```

```

        end
    end
    [fitBestIter,idxIter]=max(pf);
    if better(fitBestIter,feas(idxIter),gfit,gfeas)
        gbest=pbest(idxIter,:); gfit=fitBestIter; gfeas=feas(idxIter);
islands(s).stall=0;
    else
        islands(s).stall=islands(s).stall+1;
    end
    islands(s).X=X; islands(s).V=V; islands(s).pbest=pbest;
    islands(s).pbest_fit=pf; islands(s).pbest_feas=feas;
    islands(s).gbest=gbest; islands(s).gbest_fit=gfit;
islands(s).gbest_feas=gfeas;
    islands(s).bestHist(it)=gfit;

    if islands(s).stall>=stallMax
        n_mut=max(3,round(0.2*pop_island));
pick=randperm(pop_island,n_mut);
        noise=[0.08*pi,3.0,0.6,0.3];
        for kk=1:n_mut
            xx=gbest+randn(1,D).*noise; xx=max(lb,min(ub,xx));
            tau=xx(3); fk=xx(4); tau=min(max(tau,0),T_hit);
fk=min(max(fk,0),min(tau,fmax_k));
            xx(3)=tau; xx(4)=fk; xx(2)=min(max(xx(2),70),140);
            X(pick(kk),:)=xx; V(pick(kk),:)=0;
            [pf(pick(kk)),feas(pick(kk))] = fitness1_marginal(xx, FYk,
Qc, dt, m0, u_m, vm, vs, rs, life, T_hit, fmax_k, U_prev);
            pbest(pick(kk),:)=xx;
        end
        islands(s).X=X; islands(s).V=V; islands(s).pbest=pbest;
        islands(s).pbest_fit=pf; islands(s).pbest_feas=feas;
islands(s).stall=0;
    end
end

if mod(it,20)==0
    bests = arrayfun(@(S) S.gbest_fit, islands);
    fprintf('%3d/%3d 最佳边际增益: ', it, iterMax);
    fprintf(' '); fprintf(' %.3f', bests); fprintf(' ] | 全局
= %.3f\n', max(bests));
end

if mod(it,migPeriod)==0 && it<iterMax
    tops = cell(K,1);

```

```

        for s=1:K
            [~,ib]=max(islands(s).pbest_fit);
tops{s}=islands(s).pbest(ib,:);
        end
        for s=1:K
            to=mod(s,K)+1;
            [~,iw]=min(islands(to).pbest_fit);
            cand=tops{s};
            islands(to).X(iw,:)=cand; islands(to).V(iw,:)=0;
            [fc,fcfeas] = fitness1_marginal(cand, FYk, Qc, dt, m0, u_m, vm,
vs, rs, life, T_hit, fmax_k, U_prev);
            islands(to).pbest(iw,:)=cand; islands(to).pbest_fit(iw)=fc;
islands(to).pbest_feas(iw)=fcfeas;
            if better(fc,fcfeas, islands(to).gbest_fit,
islands(to).gbest_feas)
                islands(to).gbest=cand; islands(to).gbest_fit=fc;
islands(to).gbest_feas=fcfeas;
            end
        end
        bests = arrayfun(@(S) S.gbest_fit, islands);
    end
    [~,sBest] = max(arrayfun(@(S) S.gbest_fit, islands));
    xbest = islands(sBest).gbest;
end

```

附录 3

t_1.m

```

clear; clc;

g    = 9.8;
vd   = 300;
vg   = 3;
rg   = 10;
tg= 20;

RY = 7;
HY = 10;
CY = [0, 200];

M1 = [20000, 0, 2000];
FY1 = [17800, 0, 1800];
v = 120;

```

```

t_s = 1.5;
t_l = 3.6;
t_b = t_s + t_l;
theta = pi;
[xe, ye, ze] = explosion_point(FY1, theta, v, t_b, t_l, g);
e = [xe, ye, ze];
x = [theta, v, t_b, t_l];
u_m = - M1 / norm(M1);
T_hit = norm(M1) / vd;
t_min = t_b;
t_max = min(t_b + tg, T_hit);

Nl = 360;
Nr = 1;
Q = sample_disks(CY, RY, HY, Nl, Nr);

function_val = @(t) ( ...
    max( dist_point_to_segments( cloud_center(t, e, vg, t_b) , missile_pos(t,
M1, u_m, vd), Q ) ) - rg );

dt_c = 0.01;
ts = t_min:dt_c:t_max;
vals = arrayfun(function_val, ts);
[~, covered, out] = fitness_x(x, FY1, M1, u_m, vd, vg, rg, tg, Q, dt_c, g);

fprintf('起爆时刻 t_b = %.6f s\n', t_b);
fprintf('起爆点 e = (%.6f, %.6f, %.6f) m\n', e(1), e(2), e(3));
fprintf('有效时间窗 = [%.6f, %.6f] s\n', t_min, t_max);
if isempty(out.intervals)
    fprintf('有效遮蔽区间: 无\n');
else
    fprintf('有效遮蔽区间 (共 %d 段):\n', size(out.intervals,1));
    for i = 1:size(out.intervals,1)
        fprintf(' [% .6f, % .6f] 时长= %.6f s\n', out.intervals(i,1),
out.intervals(i,2), out.durations(i));
    end
end
fprintf('总有效遮蔽时长 T_C = %.6f s\n', out.T_C);

```

附录 3

t_2.m

clear; clc;

```

g    = 9.8;
vd   = 300;
vg   = 3;
rg   = 10;
tg   = 20;
RY   = 7;
HY   = 10;
CY   = [0,200];
M1   = [20000,0,2000];
u_m  = -M1/norm(M1);
T_hit = norm(M1)/vd;
FY1  = [17800,0,1800];
tlmax = sqrt(2*FY1(3)/g);

PSO.Nl = 12;
PSO.Nr = 1;
PSO.dt = 0.10;
HI.Nl  = 24;
HI.Nr  = 1;
HI.dt  = 0.01;

Q_c = sample_disks(CY, RY, HY, PSO.Nl, PSO.Nr);
Q_fine = sample_disks(CY, RY, HY, HI.Nl, HI.Nr);

D=4;
pop=10;
iterMax=1000;
stallMax=25;
w_max=0.9;
w_min=0.45;
c1=2.05;
c2=2.05;
phi=c1+c2;
chi=2/(phi-2+sqrt(phi^2-4*phi));
range = [2*pi, 70, T_hit, tlmax];
v_s   = 0.5*range;
lb    = [-pi, 70, 0, 0];
ub    = [ pi,140, T_hit, tlmax];
rng(1);
X = zeros(pop,D); V = zeros(pop,D);

n_rand = pop;
for i=1:n_rand

```

```

X(i,1) = (rand*2-1)*pi;
X(i,2) = 70 + 70*rand;
X(i,3) = T_hit*rand;
X(i,4) = min(X(i,3), tlmax)*rand;
end

for j=1:D, V(:,j) = (rand(pop,1)*2-1).*v_s(j); end
pbest = X;
[pbest_fit, pbest_feas] = arrayfun(@(i)
fitness_c(X(i,:),FY1,M1,u_m,vd,vg,rg,tg,Q_c,PS0.dt,g,tlmax),1:pop);
[gbest_fit, idxBest] = max(pbest_fit); gbest = pbest(idxBest,:); gbest_feas =
pbest_feas(idxBest);
bestHist = zeros(iterMax,1); stall=0;

for it=1:iterMax
w = w_max - (w_max-w_min)*(it-1)/(iterMax-1);
r1=rand(pop,D); r2=rand(pop,D);
V = chi*(w*V + c1*r1.*(pbest-X) + c2*r2.*(gbest - X));
for j=1:D, V(:,j) = max(-v_s(j), min(v_s(j), V(:,j))); end
X = X + V;
X = max(repmat(lb,pop,1), min(repmat(ub,pop,1), X));
id = X(:,4) > X(:,3); X(id,4) = X(id,3);
c1L = X <= repmat(lb,pop,1); c1H = X >= repmat(ub,pop,1);
V(c1L | c1H) = 0;
fit=zeros(pop,1); feas=false(pop,1);

for i=1:pop
[fit(i), feas(i)] =
fitness_c(X(i,:),FY1,M1,u_m,vd,vg,rg,tg,Q_c,PS0.dt,g,tlmax);
end

for i=1:pop
if better(fit(i),feas(i), pbest_fit(i), pbest_feas(i))
pbest(i,:)=X(i,:); pbest_fit(i)=fit(i); pbest_feas(i)=feas(i);
end
end
[fitBestIter, idxIter] = max(pbest_fit);
if better(fitBestIter,pbest_feas(idxBest), gbest_fit, gbest_feas)
gbest = pbest(idxBest,:); gbest_fit=fitBestIter;
gbest_feas=pbest_feas(idxBest); stall=0;
else
stall = stall+1;
end
bestHist(it)=gbest_fit;

```



```

    if stall>=stallMax
        n_mut=max(4,round(0.2*pop)); pick=randperm(pop,n_mut);
        noise=[0.08*pi, 3.0, 0.6, 0.3];
        for k=1:n_mut
            xx = gbest + randn(1,D).*noise;
            xx = max(lb, min(ub, xx));
            if xx(4)>xx(3), xx(4)=xx(3); end
            X(pick(k),:)=xx; V(pick(k),:)=0;
            [pbest_fit(pick(k)), pbest_feas(pick(k))] =
fitness_c(xx,FY1,M1,u_m,vd,vg,rg,tg,Q_c,PSO.dt,g,tlmax);
            pbest(pick(k),:)=xx;
        end
        stall=0;
    end
end

[~,~,out] = fitness_x(gbest, FY1, M1, u_m, vd, vg, rg, tg, Q_fine, HI.dt, g);
theta=gbest(1);
v=gbest(2);
tau=gbest(3);
fuse=gbest(4);
[xe,ye,ze] = explosion_point(FY1, theta, v, tau, fuse, g);

fprintf('航向角 theta = %.6f rad\n', round(theta,6));
fprintf('飞行速度 v = %.6f m/s\n', round(v,6));
fprintf('起爆时刻 tau = %.6f s\n', round(tau,6));
fprintf('起爆点 e = (%.6f, %.6f, %.6f) m\n',
round(xe,6),round(ye,6),round(ze,6));
fprintf('延迟 f = %.6f s\n', round(fuse,6));
fprintf('有效时间窗 = [%.6f, %.6f] s\n', round(out.tStart,6),
round(out.tEnd,6));
if isempty(out.intervals)
    fprintf('有效遮蔽区间: 无\n');
else
    fprintf('有效遮蔽区间 (%d 段) :\n', size(out.intervals,1));
    for i=1:size(out.intervals,1)
        fprintf('[% .6f, % .6f]时长= %.6f s\n', round(out.intervals(i,1),6),
round(out.intervals(i,2),6), round(out.durations(i),6));
    end
end
fprintf('总有效遮蔽时长 T_cov = %.6f s\n', round(out.T_C,6));

```

附录 3

t 3.m

```
clear; clc;

g = 9.8;
vd = 300;
vg = 3;
rg = 10;
tg = 20;
RY = 7;
HY = 10;
CY = [0,200];
M1 = [20000,0,2000];
u_m = -M1/norm(M1);
T_hit= norm(M1)/vd;

FY1 = [17800,0,1800];
tlmax = sqrt(2*FY1(3)/g);

Nl_disk = 120;
Nr_disk = 1;
Q = sample_disks(CY, RY, HY, Nl_disk, Nr_disk);
dt_c = 0.06;
dt_x = 0.01;

D = 8;
pop = 10;
iterMax = 1000;
stallMax = 20;
w_max=0.9;
w_min=0.5;
c1=2.0;
c2=2.0;
phi=c1+c2;
chi=2/(phi-2+sqrt(phi^2-4*phi));
lb = [-pi, 70, 0, 0, 0, 0, 0, 0];
ub = [ pi,140, T_hit, tlmax, T_hit, tlmax, T_hit, tlmax];
v_s = 0.4 * [2*pi, 70, T_hit, tlmax, T_hit, tlmax, T_hit, tlmax];

rng(99);
X = zeros(pop,D); V = zeros(pop,D);
for i=1:pop
    X(i,1) = (rand*2-1)*pi;
    X(i,2) = 70 + 70*rand;
```

```

    for j=1:3
        t_bj = T_hit*rand;
        fj = min(t_bj, tlmax)*rand;
        X(i,2*j+1) = t_bj;
        X(i,2*j+2) = fj;
    end
end
for j=1:D, V(:,j) = (rand(pop,1)*2-1).*v_s(j); end

pbest = X;
pfit = -inf(pop,1); pfeas = false(pop,1);

fprintf('开始 PSO 优化');
for i=1:pop
    [pfit(i), pfeas(i)] = fit_union(X(i,:), FY1, M1, u_m, vd, vg, rg, tg, Q,
dt_c, g, tlmax, T_hit);
end
[~, idx0] = max(pfit); gbest = pbest(idx0,:); gfit = pfit(idx0); gfeas =
pfeas(idx0);

stall=0;
for it=1:iterMax
    w = w_max - (w_max - w_min) * (it-1)/(iterMax-1);
    r1=rand(pop,D); r2=rand(pop,D);
    V = chi * ( w*V + c1*r1.*(pbest - X) + c2*r2.*( repmat(gbest,pop,1) - X) );
    for j=1:D
        V(:,j) = max(-v_s(j), min(v_s(j), V(:,j)));
    end
    X = X + V;
    X = max(repmat(lb,pop,1), min(repmat(ub,pop,1), X));
    for k=1:3
        id = X(:,2*k+2) > X(:,2*k+1);
        X(id,2*k+2) = X(id,2*k+1);
    end
    touched = (X<=repmat(lb,pop,1)) | (X>=repmat(ub,pop,1));
    V(touched) = 0;
    for i=1:pop
        [fit_i, feas_i] = fit_union(X(i,:), FY1, M1, u_m, vd, vg, rg, tg, Q,
dt_c, g, tlmax, T_hit);
        if better(fit_i,feas_i, pfit(i), pfeas(i))
            pfit(i)=fit_i; pfeas(i)=feas_i; pbest(i,:)=X(i,:);
        end
    end
    [fitIter, idx] = max(pfit);

```

```

    if better(fitIter, pfeas(idx), gfit, gfeas)
        gfit = fitIter; gbest = pbest(idx,:); gfeas = pfeas(idx); stall=0;
    else
        stall = stall + 1;
    end

    if mod(it,20)==0 || it==1
        fprintf('迭代%3d/%3d, 当前最优并集时长= %.3f s\n', it, iterMax,
max(gfit,0));
    end

    if stall>=stallMax
        pick = randperm(pop, max(4,round(0.2*pop)));
        noise = [0.05*pi, 2.0, 0.6, 0.3, 0.6, 0.3, 0.6, 0.3];
        for k=1:numel(pick)
            xx = gbest + randn(1,D).*noise;
            xx = max(lb, min(ub, xx));
            for j=1:3
                if xx(2*j+2) > xx(2*j+1), xx(2*j+2)=xx(2*j+1); end
            end
            X(pick(k),:)=xx; V(pick(k),:)=0;
            [pfit(pick(k)), pfeas(pick(k))] = fit_union(xx, FY1, M1, u_m, vd,
vg, rg, tg, Q, dt_c, g, tlmax, T_hit);
            pbest(pick(k),:) = xx;
        end
        stall=0;
        fprintf('进行高斯重启\n');
    end
end

theta = gbest(1); v = gbest(2);
t_b = [gbest(3), gbest(5), gbest(7)];
t_l = [gbest(4), gbest(6), gbest(8)];
t_l = min(t_l, tlmax);
[~, idxSort] = sort(t_b - t_l);
t_b = t_b(idxSort); t_l = t_l(idxSort);

if abs(v-140)<5e-7, v=140; end
if any(abs(t_l)<5e-7), t_l(abs(t_l)<5e-7)=0; end

intervals = cell(3,1); Tj = zeros(3,1);
drop_pts = zeros(3,3); boom_pts = zeros(3,3);
for j=1:3

```

```

    [intervals{j}, Tj(j)] = intervals_x(theta, v, t_b(j), t_l(j), FY1, M1,
u_m, vd, vg, rg, tg, Q, dt_x, g, T_hit);
    t_rel = t_b(j) - t_l(j);
    drop_pts(j,:) = [FY1(1)+v*t_rel*cos(theta), FY1(2)+v*t_rel*sin(theta),
FY1(3)];
    boom_pts(j,:) = [FY1(1)+v*t_b(j)*cos(theta), FY1(2)+v*t_b(j)*sin(theta),
FY1(3)-0.5*g*t_l(j)^2];
end

U = union_all(intervals);
T_union = round(total_len(U),6);
dir_deg = round(rad2deg(theta), 6);
dir_col = repmat(dir_deg, 3, 1);
v_col = repmat(round(v,6), 3, 1);
id_col = (1:3).';
T_col = round(Tj,6);
T_out = table( ...
    dir_col, v_col, id_col, ...
    round(drop_pts(:,1),6), round(drop_pts(:,2),6),
round(drop_pts(:,3),6), ...
    round(boom_pts(:,1),6), round(boom_pts(:,2),6),
round(boom_pts(:,3),6), ...
    T_col, ...
    'VariableNames', { ...
    '无人机运动方向', '无人机运动速度 (m/s)', '烟幕干扰弹编号', ...
    '烟幕干扰弹投放点的 x 坐标 (m)', '烟幕干扰弹投放点的 y 坐标 (m)', '烟幕干扰弹投放
点的 z 坐标 (m)', ...
    '烟幕干扰弹起爆点的 x 坐标 (m)', '烟幕干扰弹起爆点的 y 坐标 (m)', '烟幕干扰弹起爆
点的 z 坐标 (m)', ...
    '有效干扰时长 (s)'} ...
);
writetable(T_out, 'result1.xlsx', 'FileType','spreadsheet');

solFine.theta = round(theta,6);
solFine.v = round(v,6);
solFine.t_b = round(t_b(:),6);
solFine.t_l = round(t_l(:),6);
solFine.drop = round(drop_pts,6);
solFine.boom = round(boom_pts,6);
solFine.intervals = intervals;
solFine.U = U;
solFine.T_union = T_union;
rows = T_out;

```

```

fprintf('并集处理后的总有效遮蔽时长 = %.6f s\n', solFine.T_union);
for j=1:3
    ivg = solFine.intervals{j};
    if isempty(ivg)
        fprintf('弹%d: 无有效遮蔽。 \n', j);
    else
        fprintf('弹%d 遮蔽区间 (%d 段): \n', j, size(ivg,1));
        for k=1:size(ivg,1)
            fprintf(' [%.6f, %.6f] 时长= %.6f s\n', ivg(k,1), ivg(k,2),
            ivg(k,2)-ivg(k,1));
        end
    end
end
end
end

```

附录 3

t 4.m

```

clear; clc;

g = 9.8;
vd = 300;
vg = 3;
rg = 10;
life = 20;
RY = 7;
HY = 10;
CY = [0,200];

M1 = [20000, 0, 2000];
u_m = - M1 / norm(M1);
T_hit= norm(M1)/vd;

FY = [17800, 0, 1800;
      12000, 1400, 1400;
      6000, -3000, 700];
fmax = sqrt(2*FY(:,3)/g);

PSO.Ntheta = 12; PSO.Nr = 1; PSO.dt = 0.10;
HI.Ntheta = 36; HI.Nr = 1; HI.dt = 0.01;

Qc = sample_disks(CY, RY, HY, PSO.Ntheta, PSO.Nr);
Qf = sample_disks(CY, RY, HY, HI.Ntheta, HI.Nr);

order = zeros(1,3);

```

```

x_best = zeros(3,4);
iv_cell = cell(3,1);
dur_vec = zeros(3,1);
U_union = zeros(0,2);

cands = [1,2,3];
for stage = 1:3
    fprintf('\n 阶段%d: 在当前并集基础上, 选择下一架无人机\n', stage);
    best_gain = -inf; best_k = -1; best_x = []; best_iv = zeros(0,2);
    for idx = 1:numel(cands)
        k = cands(idx);
        fprintf(' 尝试 FY%d 的 4 维优化以最大化“边际增益”\n', k);
        xk = solve_one_uav_islands(FY(k,:), fmax(k), Qc, PSO.dt, ...
                                   M1, u_m, vd, vg, rg, life, T_hit, U_union);
        iv_c = intervals_one_c(xk, FY(k,:), Qc, PSO.dt, ...
                               M1, u_m, vd, vg, rg, life, T_hit);
        gain = total_len(union_all([U_union; iv_c])) - total_len(U_union);
        fprintf('FY%d 粗评边际增益≈%.3f s\n', k, gain);
        if gain > best_gain
            best_gain = gain;
            best_k = k;
            best_x = xk;
            best_iv = iv_c;
        end
    end
    order(stage) = best_k;
    x_best(best_k,:) = best_x;
    [iv_x, dur_x] = intervals_one_x(best_x, FY(best_k,:), Qf, HI.dt, ...
                                   M1, u_m, vd, vg, rg, life, T_hit, g);
    iv_cell{best_k} = iv_x;
    dur_vec(best_k) = sum(dur_x);
    U_union = union_all([U_union; iv_x]);
    fprintf('阶段%d 选择 FY%d, 细评增益 = %.6f s; 当前并集总时长 = %.6f s\n', ...
           stage, best_k, sum(max(0, iv_x(:,2)-iv_x(:,1))),
           total_len(U_union));
    cands(cands==best_k) = [];
end

for k=1:3
    fprintf('FY%d: \n', k);
    iv = iv_cell{k};
    if isempty(iv)
        fprintf('无有效遮蔽区间; 时长= 0.000000 s\n');
    else

```

```

        for i=1:size(iv,1)
            fprintf('[% .6f, % .6f]时长= %.6f s\n', iv(i,1), iv(i,2), iv(i,2)-
iv(i,1));
        end
        fprintf('合计时长= %.6f s\n', dur_vec(k));
    end
end
fprintf('并集去重后的总遮蔽时长= %.6f s\n', total_len(U_union));

theta = x_best(:,1); v = x_best(:,2); t_b = x_best(:,3); t_l = x_best(:,4);
z0 = FY(:,3);
rx=zeros(3,1); ry=zeros(3,1); rz=z0(:);
ex=zeros(3,1); ey=zeros(3,1); ez=zeros(3,1);
for k=1:3
    trel = t_b(k) - t_l(k);
    rx(k) = FY(k,1) + v(k)*trel*cos(theta(k));
    ry(k) = FY(k,2) + v(k)*trel*sin(theta(k));
    ex(k) = FY(k,1) + v(k)*t_b(k)*cos(theta(k));
    ey(k) = FY(k,2) + v(k)*t_b(k)*sin(theta(k));
    ez(k) = FY(k,3) - 0.5*g*t_l(k)^2;
end

dir_deg = mod(rad2deg(theta(:)), 360);
col1 = round(dir_deg, 6);
col3 = int32((1:3)');
col2 = round(v(:), 6);
col4 = round(rx(:), 6);
col5 = round(ry(:), 6);
col6 = round(rz(:), 6);
col7 = round(ex(:), 6);
col8 = round(ey(:), 6);
col9 = round(ez(:), 6);
col10 = round(dur_vec(:), 6);
M = [col1(:), col2(:), col4(:), col5(:), col6(:), col7(:), col8(:), col9(:),
col10(:)];
fname = 'result2.xlsx';
sheetName = 'Sheet1';
writematrix(M, fname, 'Sheet', sheetName, 'Range', 'B2');

```

附录 3

t_5.m

```

clear; clc;
USE_PARALLEL = true;

```



```

HAS_PCT = license('test','Distrib_Computing_Toolbox');
if USE_PARALLEL && HAS_PCT
    pool = gcp('nocreate');
    if isempty(pool)
        parpool('local');
        fprintf('[并行] 已启动本地并行池。\\n');
    else
        fprintf('[并行] 复用已有并行池 (%d workers)。\\n', pool.NumWorkers);
    end
else
    if ~HAS_PCT && USE_PARALLEL
        fprintf('[并行] 未检测到 Parallel Computing Toolbox, 自动回退为串行。\\n');
    else
        fprintf('[并行] 并行开关关闭, 串行执行。\\n');
    end
    USE_PARALLEL = false;
end

FAIR.alpha = 0.65;
FAIR.eps = 0.25;

g = 9.8;
vd = 300;
vg = 3;
rg = 10;
tg = 20;

RY = 7;
HY = 10;
CY = [0,200];

M0 = [ 20000, 0, 2000;
       19000, 600, 2100;
       18000, -600, 1900];
uM = - M0 ./ vecnorm(M0,2,2);
T_hit = vecnorm(M0,2,2) / vd;
T_hit_max = max(T_hit);

FY = [17800, 0, 1800;
      12000, 1400, 1400;
      6000, -3000, 700;
      11000, 2000, 1800;
      13000, -2000, 1300];

```

```

z0 = FY(:,3);
fmaxU = sqrt(2*z0/g);

PSO.Nl = 12; PSO.Nr = 1; PSO.dt = 0.08;
HI.Nl = 24; HI.Nr = 1; HI.dt = 0.01;

Qc = sample_disks(CY, RY, HY, PSO.Nl, PSO.Nr);
Qf = sample_disks(CY, RY, HY, HI.Nl, HI.Nr);
fprintf('采样点数（粗/细）： %d / %d\n', size(Qc,1), size(Qf,1));

rng(2025);
U_missile = {zeros(0,2), zeros(0,2), zeros(0,2)};
UAV_fixed = false(5,1);
UAV_theta = zeros(5,1);
UAV_v = zeros(5,1);
UAV_used = zeros(5,1);
UAV_tau_log = cell(5,1);

rows = struct('uid',0,'theta',0,'v',0,'bid',0, ...
              'rx',0,'ry',0,'rz',0, 'ex',0,'ey',0,'ez',0, ...
              'Tc',0,'mid',0, 'iv',zeros(0,2));
row_list = repmat(rows, 15, 1);
row_cnt = 0;

for step = 1:15
    fprintf('\n=== 第 %d / 15 次安排行动： 在所有可投放无人机中选“公平性评分”最大的  

    候选 ===\n', step);
    totM_pre = [total_len(U_missile{1}), total_len(U_missile{2}),
    total_len(U_missile{3})];
    if step>=5 && any(totM_pre==0)
        FAIR.alpha = min(0.9, FAIR.alpha + 0.10);
    end

    cand_gain = -inf(5,1);
    cand_uid = (1:5).';
    cand_x = cell(5,1);
    cand_mid = -ones(5,1);
    cand_iv = cell(5,1);
    cand_msg = strings(5,1);

    if USE_PARALLEL
        parfor uid = 1:5
            [cand_gain(uid), cand_x{uid}, cand_mid(uid), cand_iv{uid},
            cand_msg(uid)] = ...

```

```

        eval_one_uav_candidate(uid, UAV_used, UAV_fixed, UAV_theta,
UAV_v, UAV_tau_log, ...
                                FY, fmaxU, Qc, PSO.dt, M0, uM, vd, vg, rg,
tg, T_hit, ...
                                U_missile, T_hit_max, FAIR, USE_PARALLEL,
step);
    end
else
    for uid = 1:5
        [cand_gain(uid), cand_x{uid}, cand_mid(uid), cand_iv{uid},
cand_msg(uid)] = ...
            eval_one_uav_candidate(uid, UAV_used, UAV_fixed, UAV_theta,
UAV_v, UAV_tau_log, ...
                                    FY, fmaxU, Qc, PSO.dt, M0, uM, vd, vg, rg,
tg, T_hit, ...
                                    U_missile, T_hit_max, FAIR, USE_PARALLEL,
step);
    end
end

for uid=1:5
    if strlength(cand_msg(uid))>0
        fprintf('%s\n', cand_msg(uid));
    end
end

[best_gain, idx] = max(cand_gain);
best_uid = cand_uid(idx);
best_x    = cand_x{idx};
best_mid  = cand_mid(idx);
best_iv_coarse = cand_iv{idx};

if ~isfinite(best_gain) || best_uid < 0 || isempty(best_x)
    cand_u = find(UAV_used < 3, 1, 'first');
    if isempty(cand_u)
        fprintf(' * 警告：无可投放无人机。提前结束。\\n');
        break;
    end
    best_uid = cand_u; best_mid = 1;
    if ~UAV_fixed(best_uid)
        best_x = [0, 100, min(T_hit_max, 5), 0];
    else
        if isempty(UAV_tau_log{best_uid})
            tau_next = 0;

```

```

        else
            tau_next = UAV_tau_log{best_uid}(end) + 1;
        end
        best_x = [UAV_theta(best_uid), UAV_v(best_uid), min(T_hit_max,
tau_next), 0];
    end
    best_gain = 0;
    fprintf(' * 退化选择: FY%d, 公平评分=0.\n', best_uid);
end

theta = best_x(1); v = best_x(2); t_b = best_x(3); t_l = best_x(4);
uid = best_uid; mid = best_mid;

if ~UAV_fixed(uid)
    UAV_fixed(uid) = true; UAV_theta(uid) = theta; UAV_v(uid) = v;
    fprintf(' √ 锁定 FY%d 的航向与速度: theta=%.6f rad, v=%.6f m/s\n', uid,
round(theta,6), round(v,6));
end
if isempty(UAV_tau_log{uid})
    UAV_tau_log{uid} = t_b;
else
    t_b = max(t_b, UAV_tau_log{uid}(end)+1.0);
    UAV_tau_log{uid}(end+1,1) = t_b;
end
UAV_used(uid) = UAV_used(uid) + 1;
bid = UAV_used(uid);

Tc_one = 0; iv_fine_primary = zeros(0,2);
for mid_i = 1:3
    [iv_fine_i, dur_fine_i] = intervals_one_x([theta,v,t_b,t_l],
FY(uid,:), ...
                                Qf, HI.dt, M0(mid_i,:), uM(mid_i,:), vd, vg, rg,
tg, T_hit(mid_i), g);
    U_missile{mid_i} = union_all([U_missile{mid_i}; iv_fine_i]);
    if mid_i == mid
        Tc_one = sum(dur_fine_i);
        iv_fine_primary = iv_fine_i;
    end
end
iv_fine = iv_fine_primary;

trel = t_b - t_l;
rx = FY(uid,1) + v*trel*cos(theta);
ry = FY(uid,2) + v*trel*sin(theta);

```

```

    rz = FY(uid,3);
    ex = FY(uid,1) + v*t_b*cos(theta);
    ey = FY(uid,2) + v*t_b*sin(theta);
    ez = FY(uid,3) - 0.5*g*t_l^2;

    fprintf('  FY%d-弹%d -> 针对 M%d: \n', uid, bid, mid);
    if isempty(iv_fine)
        fprintf('    无有效遮蔽区间; 时长= 0.000000 s\n');
    else
        for tseg=1:size(iv_fine,1)
            fprintf('      [% .6f, % .6f] 时长= %.6f s\n', iv_fine(tseg,1),
iv_fine(tseg,2), iv_fine(tseg,2)-iv_fine(tseg,1));
        end
        fprintf('    合计时长= %.6f s\n', Tc_one);
    end

    row_cnt = row_cnt + 1;
    row_list(row_cnt).uid = uid;
    row_list(row_cnt).theta = theta;
    row_list(row_cnt).v = v;
    row_list(row_cnt).bid = bid;
    row_list(row_cnt).rx = rx; row_list(row_cnt).ry = ry;
    row_list(row_cnt).rz = rz;
    row_list(row_cnt).ex = ex; row_list(row_cnt).ey = ey;
    row_list(row_cnt).ez = ez;
    row_list(row_cnt).Tc = Tc_one;
    row_list(row_cnt).mid = mid;
    row_list(row_cnt).iv = iv_fine;

    totM = [total_len(U_missile{1}), total_len(U_missile{2}),
total_len(U_missile{3})];
    fprintf('  >>> 当前并集总时长: M1=%.3f s, M2=%.3f s, M3=%.3f s | SUM=%.3f
s\n', totM(1), totM(2), totM(3), sum(totM));
end

if row_cnt < 15
    for k=row_cnt+1:15
        row_list(k).uid = ceil(k/3); row_list(k).bid = mod(k-1,3)+1;
        row_list(k).theta=0; row_list(k).v=100;
        row_list(k).rx=0; row_list(k).ry=0; row_list(k).rz=0;
        row_list(k).ex=0; row_list(k).ey=0; row_list(k).ez=0;
        row_list(k).Tc=0; row_list(k).mid=1; row_list(k).iv=zeros(0,2);
    end
end
end

```

```

A = struct2table(row_list(1:15));
A = sortrows(A, {'uid', 'bid'});

col1 = round(A.uid,6);
col2 = round(A.theta,6);
col3 = round(A.v,6);
col4 = round(A.bid,6);
col5 = round(A.rx,6); col6=round(A.ry,6); col7=round(A.rz,6);
col8 = round(A.ex,6); col9=round(A.ey,6); col10=round(A.ez,6);
col11 = round(A.Tc,6);
col12 = round(A.mid,6);

T = table(col1,col2,col3,col4,col5,col6,col7,col8,col9,col10,col11,col12, ...
    'VariableNames', {'无人机编号','无人机运动方向','无人机运动速度 (m/s)','烟幕干
    扰弹编号', ...
        '烟幕干扰弹投放点的 x 坐标 (m)','烟幕干扰弹投放点的 y 坐标
    (m)','烟幕干扰弹投放点的 z 坐标 (m)', ...
        '烟幕干扰弹起爆点的 x 坐标 (m)','烟幕干扰弹起爆点的 y 坐标
    (m)','烟幕干扰弹起爆点的 z 坐标 (m)', ...
        '有效干扰时长 (s)','干扰的导弹编号'});
writetable(T, 'result3.xlsx');
fprintf('\n 完成: result3.xlsx 已生成 (15 行×12 列)。 \n');

totM = [total_len(U_missile{1}), total_len(U_missile{2}),
total_len(U_missile{3})];
fprintf('最终并集总时长: M1=%.6f s, M2=%.6f s, M3=%.6f s | SUM=%.6f s\n',
totM(1), totM(2), totM(3), sum(totM));
fprintf('=== 结束 ===\n');

function [gain_best, x_best, mid_best, iv_best, msg] =
eval_one_uav_candidate(uid, UAV_used, UAV_fixed, UAV_theta, UAV_v,
UAV_tau_log, ...
    FY, fmaxU, Qc, dt, M0, uM, vd, vg, rg, tg, T_hit, U_missile, T_hit_max,
FAIR, USE_PARALLEL, step)

    gain_best = -inf; x_best = []; mid_best = -1; iv_best = zeros(0,2); msg =
    '';

    if UAV_used(uid) >= 3
        return;
    end
    rng( 12345 + 97*uid + 13*step + sum(UAV_used) );

```

```

    if ~UAV_fixed(uid)
        [x_cand, gain_cand, mid_cand, iv_cand] = ...
            pso_first_bomb(uid, FY(uid,:), fmaxU(uid), ...
                Qc, dt, M0, uM, vd, vg, rg, tg, T_hit, ...
                U_missile, FAIR, USE_PARALLEL);

        msg = sprintf(' FY%d 首弹候选: 公平评分 ≈ %.3f (主攻 M%d)', uid,
            gain_cand, mid_cand);
        if gain_cand > gain_best
            gain_best = gain_cand; x_best = x_cand; mid_best = mid_cand;
            iv_best = iv_cand;
        end
    else
        tau_lb = 0;
        if ~isempty(UAV_tau_log{uid})
            tau_lb = UAV_tau_log{uid}(end) + 1.0;
        end
        if tau_lb >= T_hit_max
            msg = sprintf(' FY%d 可投放时窗已尽 (tau_lb>=T_hit_max), 跳过.',
                uid);
            return;
        end

        [y_cand, gain_cand, mid_cand, iv_cand] = ...
            pso_next_bomb(uid, FY(uid,:), UAV_theta(uid), UAV_v(uid),
                fmaxU(uid), tau_lb, ...
                Qc, dt, M0, uM, vd, vg, rg, tg, T_hit, ...
                U_missile, FAIR, USE_PARALLEL);

        msg = sprintf(' FY%d 后续弹候选: 公平评分 ≈ %.3f (主攻 M%d)', uid,
            gain_cand, mid_cand);
        if gain_cand > gain_best
            gain_best = gain_cand; x_best = [UAV_theta(uid), UAV_v(uid),
                y_cand(1), y_cand(2)];
            mid_best = mid_cand; iv_best = iv_cand;
        end
    end
end

function [gain_sum, mid_star, iv_star, gain_vec] = marginal_all_of_3(x, FYu,
    Q, dt, M0, uM, vd, vg, rg, tg, T_hit, U_missile)
    gain_vec = zeros(1,3);
    iv_keep = cell(1,3);
    for mid=1:3

```

```

        iv = intervals_one_c(x, FYu, Q, dt, M0(mid,:), uM(mid,:), vd, vg, rg,
tg, T_hit(mid));
        Unew = union_all([U_missile{mid}; iv]);
        gain_vec(mid) = total_len(Unew) - total_len(U_missile{mid});
        iv_keep{mid} = iv;
    end
    gain_sum = sum(gain_vec);
    [~, mid_star] = max(gain_vec);
    iv_star = iv_keep{mid_star};
end

function fit = fitness4_fair(x, FYu, Q, dt, M0, uM, vd, vg, rg, tg, T_hit,
fmax_u, U_missile, FAIR)
    theta=x(1); v=x(2); t_b=x(3); t_l=x(4);
    if v<70 || v>140, fit=-1e9; return; end
    if t_b<0 || t_b>max(T_hit), fit=-1e9; return; end
    if t_l<0 || t_l>min(t_b,fmax_u), fit=-1e9; return; end
    if FYu(3) - 0.5*9.8*t_l^2 < 0, fit=-1e9; return; end

    L_old = [total_len(U_missile{1}), total_len(U_missile{2}),
total_len(U_missile{3})];
    gain_sum = 0;
    util_inc = 0;
    for mid=1:3
        iv = intervals_one_c(x, FYu, Q, dt, M0(mid,:), uM(mid,:), vd, vg, rg,
tg, T_hit(mid));
        Unew = union_all([U_missile{mid}; iv]);
        L_new = total_len(Unew);
        gain = L_new - L_old(mid);
        gain_sum = gain_sum + gain;
        util_inc = util_inc + ( log(FAIR.eps + L_new) - log(FAIR.eps +
L_old(mid)) );
    end

    fit = (1-FAIR.alpha)*gain_sum + FAIR.alpha*util_inc;
    if ~isfinite(fit), fit = -1e9; end
end

function [xbest, gain_best, mid_best, iv_best] = pso_first_bomb(uid, FYu,
fmax_u, ...
                                Q, dt, M0, uM, vd, vg, rg, tg, T_hit, U_missile,
FAIR, USE_PARALLEL)
    D=4; pop=20; iterMax=1000; stallMax=30;
    w_max=0.9; w_min=0.5; c1=2.0; c2=2.0;

```



```

phi=c1+c2; chi=2/(phi-2+sqrt(max(phi^2-4*phi,1e-12)));
lb=[-pi,70,0,0]; ub=[pi,140,max(T_hit),fmax_u];
vmax=0.4*[2*pi,70,max(T_hit),fmax_u];

rng(100+uid);
X=zeros(pop,D); V=zeros(pop,D);
for i=1:pop
    X(i,1)=(rand*2-1)*pi; X(i,2)=70+70*rand;
    X(i,3)=max(T_hit)*rand; X(i,4)=rand*min(X(i,3),fmax_u);
end
for j=1:D, V(:,j)=(rand(pop,1)*2-1).*vmax(j); end

pbest=X; pfit=-inf(pop,1);

if USE_PARALLEL
    parfor i=1:pop
        pfit(i) = fitness4_fair(X(i,:), FYu, Q, dt, M0, uM, vd, vg, rg, tg,
T_hit, fmax_u, U_missile, FAIR);
    end
else
    for i=1:pop
        pfit(i) = fitness4_fair(X(i,:), FYu, Q, dt, M0, uM, vd, vg, rg, tg,
T_hit, fmax_u, U_missile, FAIR);
    end
end

[gbest_fit,ib]=max(pfit); gbest=X(ib,:); stall=0;

for it=1:iterMax
    w=w_max-(w_max-w_min)*(it-1)/(iterMax-1);
    r1=rand(pop,D); r2=rand(pop,D);
    V = chi*( w*V + c1*r1.*(pbest-X) + c2*r2.*( repmat(gbest,pop,1)-X) );
    for j=1:D, V(:,j)=max(-vmax(j),min(vmax(j),V(:,j))); end
    X = X + V;
    for i=1:pop
        xi=X(i,:); xi=max(lb,min(ub,xi));
        t_b=xi(3); fk=xi(4); t_b=min(max(t_b,0),max(T_hit));
fk=min(max(fk,0),min(t_b,fmax_u));
        xi(3)=t_b; xi(4)=fk; xi(2)=min(max(xi(2),70),140); X(i,:)=xi;
    end
    touched=(X<=repmat(lb,pop,1))|(X>=repmat(ub,pop,1)); V(touched)=0;

    fit=zeros(pop,1);

```

```

    if USE_PARALLEL
        parfor i=1:pop
            fit(i) = fitness4_fair(X(i,:), FYu, Q, dt, M0, uM, vd, vg, rg,
tg, T_hit, fmax_u, U_missile, FAIR);
        end
    else
        for i=1:pop
            fit(i) = fitness4_fair(X(i,:), FYu, Q, dt, M0, uM, vd, vg, rg,
tg, T_hit, fmax_u, U_missile, FAIR);
        end
    end

    for i=1:pop
        if fit(i) > pfit(i)
            pbest(i,:)=X(i,:); pfit(i)=fit(i);
        end
    end
    [fitBestIter,idxIter]=max(pfit);
    if fitBestIter > gbest_fit
        gbest=pbest(idxIter,:); gbest_fit=fitBestIter; stall=0;
    else
        stall=stall+1;
    end
    if stall>=30
        n_mut=max(4,round(0.2*pop)); pick=randperm(pop,n_mut);
        noise=[0.08*pi,3.0,0.6,0.3];
        for kk=1:n_mut
            xx=gbest+randn(1,D).*noise; xx=max(lb,min(ub,xx));
            t_b=xx(3); fk=xx(4); t_b=min(max(t_b,0),max(T_hit));
fk=min(max(fk,0),min(t_b,fmax_u));
            xx(3)=t_b; xx(4)=fk; xx(2)=min(max(xx(2),70),140);
            X(pick(kk),:)=xx; V(pick(kk),:)=0;
            pfit(pick(kk)) = fitness4_fair(xx, FYu, Q, dt, M0, uM, vd, vg,
rg, tg, T_hit, fmax_u, U_missile, FAIR);
            pbest(pick(kk),:)=xx;
        end
        stall=0;
    end
end

xbest = gbest;
[ gain_best, mid_best, iv_best] = marginal_all_of_3(xbest, FYu, Q, dt, M0,
uM, vd, vg, rg, tg, T_hit, U_missile);
end

```

```

function [ybest, gain_best, mid_best, iv_best] = pso_next_bomb(uid, FYu,
theta, v, fmax_u, tau_lb, ...
                                Q, dt, M0, uM, vd, vg, rg, tg, T_hit, U_missile,
FAIR, USE_PARALLEL)
    D=2; pop=36; iterMax=130; stallMax=25;
    w_max=0.9; w_min=0.35; c1=1.8; c2=1.8;
    phi=c1+c2; chi=2/(phi-2+sqrt(max(phi^2-4*phi,1e-12)));
    lb=[tau_lb, 0]; ub=[max(T_hit), fmax_u];
    vmax=0.5*[max(T_hit)-tau_lb, fmax_u];

    rng(500+uid);
    X=zeros(pop,D); V=zeros(pop,D);
    for i=1:pop
        tau_i = tau_lb + (max(T_hit)-tau_lb)*rand;
        fi = rand * min(tau_i, fmax_u);
        X(i,:)=tau_i, fi];
    end
    for j=1:D, V(:,j)=(rand(pop,1)*2-1).*vmax(j); end

    pbest=X; pfit=-inf(pop,1);

    if USE_PARALLEL
        parfor i=1:pop
            pfit(i) = fitness4_fair([theta,v,X(i,1),X(i,2)], FYu, Q, dt, M0,
uM, vd, vg, rg, tg, T_hit, fmax_u, U_missile, FAIR);
        end
    else
        for i=1:pop
            pfit(i) = fitness4_fair([theta,v,X(i,1),X(i,2)], FYu, Q, dt, M0,
uM, vd, vg, rg, tg, T_hit, fmax_u, U_missile, FAIR);
        end
    end

    [gbest_fit,ib]=max(pfit); gbest=X(ib,:); stall=0;

    for it=1:iterMax
        w=w_max-(w_max-w_min)*(it-1)/(iterMax-1);
        r1=rand(pop,D); r2=rand(pop,D);
        V = chi*( w*V + c1*r1.*(pbest-X) + c2*r2.*( repmat(gbest,pop,1)-X) );
        for j=1:D, V(:,j)=max(-vmax(j),min(vmax(j),V(:,j))); end
        X = X + V;

        for i=1:pop

```

```

        t_b=X(i,1); f=X(i,2);
        t_b = min(max(t_b, tau_lb), max(T_hit));
        f = min(max(f,0), min(t_b, fmax_u));
        X(i,:)= [t_b,f];
    end
    touched=(X<=repmat(lb,pop,1))|(X>=repmat(ub,pop,1)); V(touched)=0;

    fit=zeros(pop,1);

    if USE_PARALLEL
        parfor i=1:pop
            fit(i) = fitness4_fair([theta,v,X(i,1),X(i,2)], FYu, Q, dt, M0,
uM, vd, vg, rg, tg, T_hit, fmax_u, U_missile, FAIR);
        end
    else
        for i=1:pop
            fit(i) = fitness4_fair([theta,v,X(i,1),X(i,2)], FYu, Q, dt, M0,
uM, vd, vg, rg, tg, T_hit, fmax_u, U_missile, FAIR);
        end
    end

    for i=1:pop
        if fit(i) > pfit(i)
            pbest(i,:)=X(i,:); pfit(i)=fit(i);
        end
    end
    [fitBestIter,idxIter]=max(pfit);
    if fitBestIter > gbest_fit
        gbest=pbest(idxIter,:); gbest_fit=fitBestIter; stall=0;
    else
        stall=stall+1;
    end
    if stall>=stallMax
        n_mut=max(3,round(0.2*pop)); pick=randperm(pop,n_mut);
        noise=[0.6,0.3];
        for kk=1:n_mut
            xx=gbest+randn(1,D).*noise;
            t_b=xx(1); f=xx(2);
            t_b = min(max(t_b, tau_lb), max(T_hit));
            f = min(max(f,0), min(t_b, fmax_u));
            X(pick(kk),:)= [t_b,f]; V(pick(kk),:)=0;
            pfit(pick(kk)) = fitness4_fair([theta,v,t_b,f], FYu, Q, dt, M0,
uM, vd, vg, rg, tg, T_hit, fmax_u, U_missile, FAIR);
            pbest(pick(kk),:)= [t_b,f];
        end
    end
end

```

```

        end
        stall=0;
    end
end

ybest = gbest;
[ gain_best, mid_best, iv_best] =
marginal_all_of_3([theta,v,ybest(1),ybest(2)], FYu, Q, dt, M0, uM, vd, vg, rg,
tg, T_hit, U_missile);
end

```

附录 3

total_len.m

```

function L = total_len(iv)
    if isempty(iv), L=0; else, L = sum(max(0, iv(:,2)-iv(:,1))); end
end

```

附录 3

union_all.m

```

function U = union_all(intervals)
if iscell(intervals)
    all = zeros(0,2);
    for i=1:numel(intervals)
        if ~isempty(intervals{i})
            all = [all; intervals{i}];
        end
    end
else
    all = intervals;
end

if isempty(all)
    U = zeros(0,2);
    return;
end

all = sortrows(all,1);
U = all(1,:);
for k = 2:size(all,1)
    if all(k,1) <= U(end,2) + 1e-9
        U(end,2) = max(U(end,2), all(k,2));
    else

```

```
        U(end+1,:) = all(k,:);  
    end  
end  
end
```